COMSATS University Islamabad, Lahore Campus
Department of Electrical & Computer Engineering
1.5KM Defence Road, Off Raiwind Road, Lahore

EEE 324 – Digital Signal Processing
Lab Manual for Spring 2024 & Onwards

Lab Resource Person

Engr. Tabassum Nawaz Bajwa

Theory Resource Person

Dr. Khurram Ali

Name: _____Registration Number: *CIIT/____-____-____/LHR*

Program: _____ Batch:

Semester

Revision History

S.No.	Update	Date	Performed by
1	Lab Manual Preparation	Sep-14	Mr. M. Usman Iqbal
2	Lab Manual Review	Feb-15	Dr. Hafiz M Asif Dr. Khurram Ali
3	Layout Modifications	Feb-15	Mr. M Usman Iqbal
4	Lab Manual Modifications	Nov-15	Mr. Haad Akmal Mr. Abdul Quddious Mr. Jawad Ali
5	Lab Manual Modifications	Aug-16	Engr. Maida Farooq Engr. Jawad Ali
6	Lab Manual Modifications	Aug-17	Mr. Mukhtar Hussain
7	Lab Manual Modifications	Nov-17	Mr. M Usman Iqbal Mr. Mukhtar Hussain
8	Lab Manual Modifications	Nov-2018	Mr.Abu bakar Talha
9	Lab Manual Modifications	Feb-2020	Mr.Abu bakar Talha
10	Lab Manual Modifications	Dec-2021	Mr.Assad Ali
11	Lab Manual Modifications	Feb-2022	Mr.Abubakar Talha
12	Lab Manual Modifications	Feb-2023	Mr.Abubakar Talha

Preface

This course builds on the basic concepts learned in the course ‘Signal and Systems', with emphasis on `digital and discrete-time signals'. The realm of DSP extends over a multitude of disciplines, ranging from communications, space exploration, computer vision, remote sensing, and even entertainment. This course, therefore, holds vital importance. Moreover, a sound knowledge of DSP is imperative in undertaking numerous other courses, offered in the subsequent semesters during the course of the study towards the award of Bachelors in Electrical/Computer engineering.

Following are the major topics covered in this course.

- Digital Signals in Time and Frequency Domains
- Discrete Time LTI Systems
- LTI Systems Analysis in Time and Frequency Domains and stability
- Z-Transform
- Sampling and Quantization of Signals
- A/D and D/A Signal Transformation
- Frequency domain analysis of signals and systems
- Discrete Fourier Transform and Fast Fourier Transform
- Digital Filters, IIR, FIR

Books

Text Books

1. Digital Signal Processing: Principles, Algorithms and Applications by John G. Proakis and Dimitris Manolakis (P&M)

Reference Books

1. Discrete-Time Signal Processing by Alan V. Oppenheim, Ronald. W. Schafer, Pearson Education (OS)
2. Signals and Systems by Alan V. Oppenheim, Alan S. Willsky and with S. Hamid (OW&N)
3. Digital Signal Processing and Applications with the C6713 and C6416 DSK by Rulph Chassaing

Reference Books for Manual

1. Digital Signal Processing: A Computer-based approach by Sanjit K. Mitra, McGraw-Hill Science (SKM)
2. Digital Signal Processing using MATLAB by Vinay K. Ingle, Jhon G. Proakis.
3. Digital Signal Processing Fundamentals and Applications by Li-Tan, Jean Jiang

Learning Outcomes

Theory CLOs

After successfully completing this course, the students will be able to:

1. Apply all the concepts of the signal and systems, their mathematical description/ representation and transformations in discrete domain to understand and analyze discrete time LTI systems. (PLO1-C3)
2. Analyze the discrete systems using Fast Fourier Transform (FFT) and filter realization techniques for discrete time signal processing. (PLO2-C4)
3. Design the digital filter using analog and digital techniques for discrete time signal processing (PLO3- C5)

Lab CLOs

After successfully completing this course, the students will be able to:

1. Follow the concepts of digital signal processing using software and hardware tools.(PLO5-P3)
2. Discuss the concepts of digital signal processing by justifying the open ended lab task using report/presentation. (PLO10-A2)

CLOs – PLOs Mapping

CLO \ Lab	Lab 1	Lab 2	Lab 3	Lab 4	Lab 5	Lab 6	Lab 7	Lab 8	Lab 9	Lab 10	Lab 11	Lab 12
CLO4	P2	P2	P2	P2	P2	P2	P3	P3	P3	P2	P3	P3
CLO5											A2	

Grading Policy

The final marks for lab would comprise of Lab Assignments (25%), Mid-Term (25%) and Lab Terminal (50%).

Mid-Term Lab Mid Term = 0.5*(Lab Mid Term) + 0.5*(average of lab evaluation of Lab 1-7)

Terminal 0.1*[(OEP marks out of 25)*2] +0.4*(Terminal Exam result out of 50) +0.25*[(average of lab evaluation of Lab 9-12) *5] + 0.10*[(average of lab evaluation of Lab 5-8) *5] +0.15*[(average of lab evaluation of Lab 1-4) *5]

Lab Assignment Marks:

- Lab Assignment 1 marks = Lab evaluation marks from experiment 1-3.
- Lab Assignment 2 marks = Lab evaluation marks from experiment 4-6.
- Lab Assignment 3 marks = Lab evaluation marks from experiment 7-9.
- Lab Assignment 4 marks = Lab evaluation marks from experiment 10-12.

The minimum pass marks for both lab and theory shall be 50%. Students obtaining less than 50% marks (in either theory or lab, or both) shall be deemed to have failed in the course. The final marks would be computed with 75% weight to theory and 25% to lab final marks.

List of Equipment

- TMS320C6713 DSP Starter Kit (DSK)

Software Resources

- MATLAB
- CODE COMPOSER STUDIO (CCS)
- LABVIEW

Lab Instructions

- This lab activity comprises of three parts: Pre-lab tasks, Lab Tasks, Conclusion and Viva
- The students should perform and demonstrate each lab task separately for step-wise evaluation.
- Only those tasks that are completed during the allocated lab time will be credited to the students.
- Students are however encouraged to practice on their own in spare time for enhancing their skills.

Instructions for open ended Lab Report

All questions should be answered precisely to get maximum credit. Lab report must ensure following items:

- Lab objectives
- MATLAB codes
- Results (graphs/tables) duly commented and discussed
- Conclusion

Safety Instructions

1. Conduct yourself in a responsible manner at all times in the laboratory.
2. Know the location of electrical panels and disconnect switches in or near your laboratory so that power can be quickly shut down in the event of a fire or electrical accident
3. Report any damages to equipment, hazards, and potential hazards to the laboratory instructor/staff available at the time.
4. If in doubt about electrical safety, see the laboratory instructor.
5. Make sure that the computers should be properly logged off when you complete your lab task.
6. Make sure that the software used in Lab is properly handled.
7. During any problem regarding software consult the Lab instructor/staff available.
8. Do not eat food, drink beverages or chew gum in the laboratory and do not use laboratory glassware as containers for food or beverages. Smoking is strictly prohibited in lab area.
9. Do not plug in a flash drive on lab computers.
10. Do not upload, delete or alter any software on the lab PC.

Table of Contents

CLO- Lab Experiment Mapping

PLO / CLOs	Lab #1	Lab #2	Lab #3	Lab #4	Lab #5	Lab #6	Lab #7	Lab #8	Lab #9	Lab #10	Lab #11	Lab #12
CLO4	x	x	x	x	x	x	x	x	x	x	x	x
CLO5											x	

LAB # 1
To sketch the basic Discrete Time Signals for Digital Signal Processing using MATLAB

Objectives

- To explain basics of signal processing using MATLAB
- To explain user defined functions and its significance using MATLAB

Pre-Lab

1. Revising the previous concepts

Here we briefly describe the basic sequence operation and their MATLAB equivalents.

1. Signal Addition

This is sample by sample addition given by

$$x_1(n) + x_2(n) = \{ x_1(n) + x_2(n)\}$$

It is implemented in MatLab by the arithmetic operation "+". However the length of the sequence $x_1(n)$ *and* $x_2(n$) should be same. If the sequences are of unequal lengths, or if the sample positions are different for the equal length sequences then we cannot directly use the operator "+". So while adding such sequences, MatLab's indexing operation requires attention.

2. Signal Multiplication

This is sample by sample multiplication or "dot" multiplication given by

$$\{ x_1(n)\}.\{x_2(n)\} = \{x_1(n)x_2(n)\}$$

It is implemented in MatLab by the array operator ".*". Once again similar restrictions apply for the .* operator as for the "+" operator. Therefore we have to develop the MatLab function 'signal_multi', which is similar to the 'signal_add'.

3. Scaling

It this operation each sample is multiplied by a scalar 'α'

$$\alpha\{x(n)\} = \{\alpha x(n)\}$$

An arithmetic operation * is used to implement this operation.

4. Shifting

In this operation, each sample of $x(n)$ is shifted by an amount of k to obtain a shifted sequence $y(n)$

$$y(n) = \{x(n - k)\}$$

if we *let* $m = n - k$, the $n = m + k$ and the above operation is given by

$$y(m + k) = \{x(m)\}$$

Hence this operation has no effect on the vector x, but the vector n is changed by adding k to each element.

5. Folding

In this operation each sample of $x(n)$ is flipped around $n = 0$ to obtain a folded sequence $y(n)$.

$$y(n) = \{ x(-n)\}$$

6. Sample Summation

This operation differs from signal addition operation. It adds all sample values of $x(n)$ between n_1 *and* n_2.

$$\sum_{n=n_1}^{n_2} x(n) = x(n_1) + \cdots + x(n_2)$$

It is implemented by the $sum(x(n1: n2))$ function.

7. Sample Products

This operation also differs from signal multiplication operation. It multiplies all sample values of $x(n)$ between n_1 and n_2.

$$\prod_{n1}^{n2} x(n) = x(n_1)\, X \ldots\ldots\ldots x(n_2)$$

It is implemented by the $prod(x(n_1: n_2)$

8. Signal Energy

The energy of a sequence $x(n)$ is given by

$$\varepsilon_x = \sum_{-\infty}^{\infty} x(n)\, x^*(n) = \sum_{-\infty}^{\infty} |x(n)|^2$$

Where superscript * denotes the operation of complex conjugation. The energy of finite duration sequence $x(n)$ can be computed in MatLab using

```
>> Ex = sum (x .* conj (x)) ;        % one approach
>> Ex = sum (abs (x) .^2) ;          % another approach
```

9. Signal Power

The average power of a periodic sequence with fundamental period N is given by

$$P_x = \frac{1}{N} \sum_{0}^{N-1} |\, x(n)|^2$$

Pre-Lab Task

Task 1

Generate and plot each of the following sequences over the interval $-n : n$, determined by your roll number as shown below.

$$n = \frac{Class\ Roll\ Number}{2} + 5$$

a. $x_1(n) = 2\ \delta(n + 2) - \delta(n - 4)$,

b. $x_2(n) = n\ [u(n) - u(n - 10)] + 10\ e^{-0.3(n-10)}\ [u(n - 10) - u(n - 20)]$

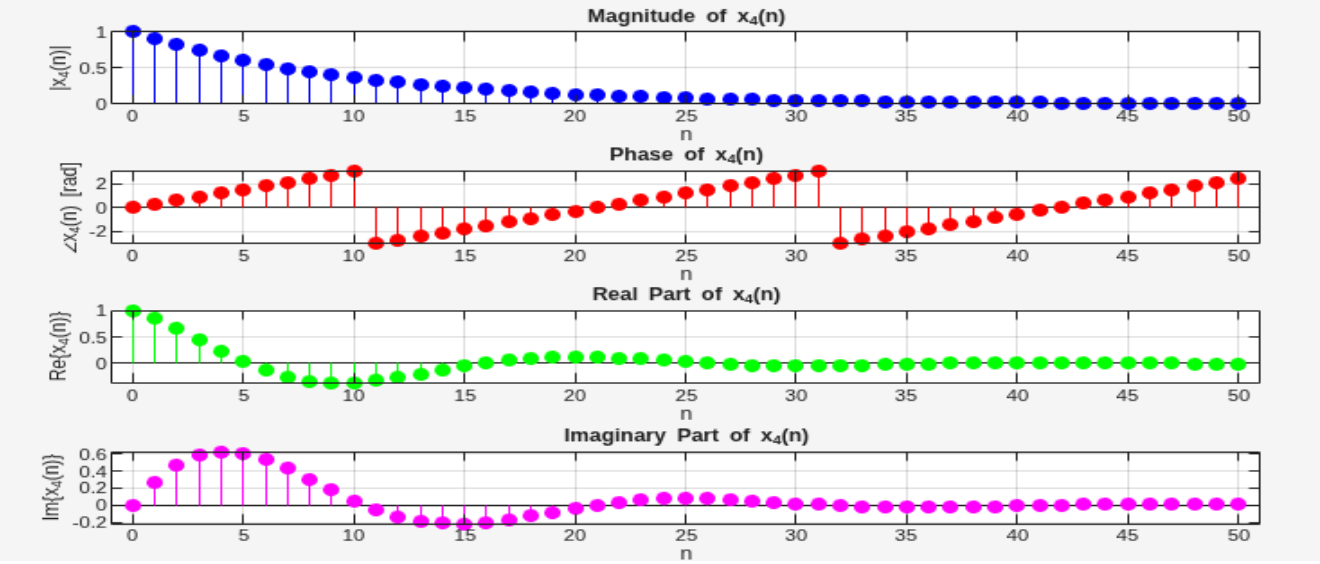
c. $x_3(n) = x_1(n) * x_2(n) + x_1(-n) * x_2(-n)$

Task 2

Generate the following complex valued signal and plot its magnitude, phase and the real part and the imaginary part in four separate subplots.

$$x_4(n) = e^{(-0.1+j0.3)n}$$

Where the value of n is same as in the lab task 1.



2. Use of Help Command

>> help 'command'

which will provide information on the inputs, outputs, usage, and functionality of the 'command'. A complete listing of commands sorted by functionality can be obtained by typing help at the prompt.

Functions

A function is a group of statements that together perform a task. Functions operate on variables within their own workspace, which is also called the local workspace, separate from the workspace you access at the MATLAB command prompt which is called the base workspace. Functions can accept more than one input arguments and may return more than one output arguments.

Use **help** command to find out how you can create and call the function. Explain it in your words

3. Plotting using MATLAB

Plot the following two signals together in the same plot.

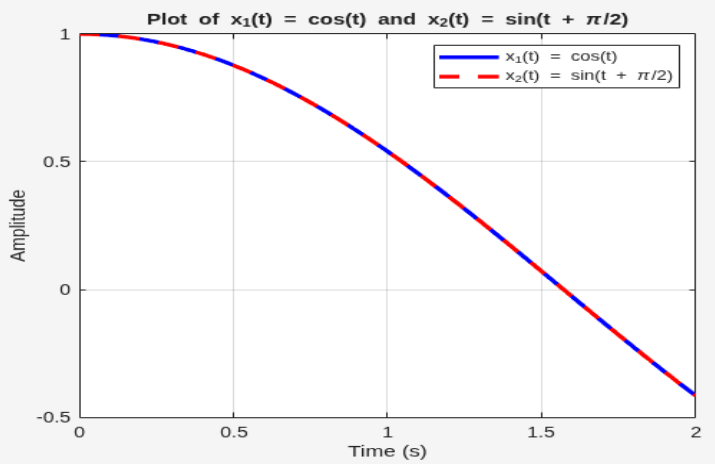
$$x_1(t) = \cos(t) \quad and \quad x_2(t) = \sin(t + \frac{\pi}{2})$$

$$where\ t = 0 \rightarrow 2sec$$

Use proper indicators, legends, title and label the axis. Use help command if require

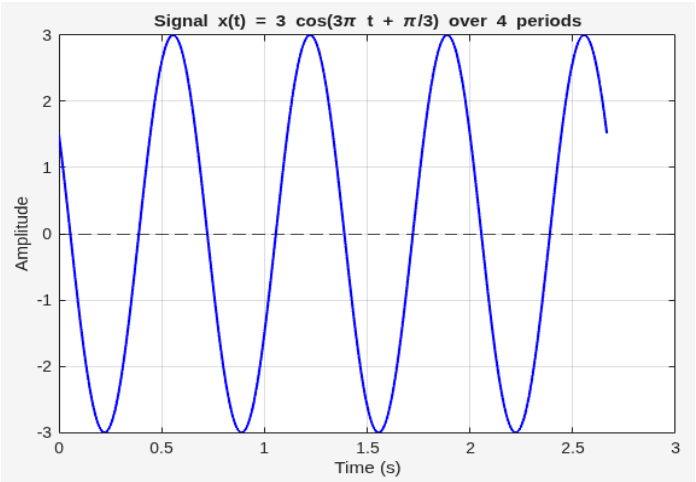
Write your MATLAB code

```
t = 0:0.01:2;
x1 = cos(t);
x2 = sin(t + pi/2);
figure;
plot(t, x1, 'b-', 'LineWidth', 2.5); hold on;
plot(t, x2, 'r--', 'LineWidth', 2.5);
xlabel('Time (s)');
ylabel('Amplitude');
title('Plot of x_1(t) = cos(t) and x_2(t) = sin(t + \
pi/2)');
legend('x_1(t) = cos(t)', 'x_2(t) = sin(t + \pi/2)');
grid on;
```



Plot the signal $x(t) = 3\cos(3\pi t + \pi/3)$ in four periods.

```
   Parameters
A = 3;              % Amplitude
w = 3*pi;           % Angular frequency
phi = pi/3;         % Phase shift
T = 2*pi/w;         % Time period
t = 0:0.001:4*T;
   Signal
x = A*cos(w*t + phi);
   Plot
figure;
plot(t, x, 'b', 'LineWidth', 1.5); hold on;
yline(0, 'k--'); % x-axis reference
```



Lab Tasks

Lab Task 1: Creating a user defined Function

Write a MATLAB functions for following basic sequences. Function should take input variables n_o, n_1, n_2 and returns 'x' and 'n'. As given by equations

1. Unit Sample Sequence

$$\delta(n - n_o) = \begin{cases} 1, & n = n_o \\ 0, & n \neq n_o \end{cases}, \text{where } n_1 \leq n \leq n_2$$

2. Unit Step Sequence

$$u(n - n_o) = \begin{cases} 1, & n \geq n_o \\ 0, & n < n_o \end{cases}, \text{where } n_1 \leq n \leq n_2$$

3. Exponential Growing Sequence

$$x(n - n_o) = e^{(n-n_o)}, \text{where } n_1 \leq n \leq n_2$$

4. Exponential Decaying Sequence

$$x(n - n_o) = e^{-(n-n_o)}, \text{where } n_1 \leq n \leq n_2$$

All plots should be properly commented, titled, labelled. All three different signals corresponding to same basic sequence should be plotted in one figure and should be differentiated by different colors and other methods.

Solution

Function 1 MATLAB Code

```
function [x,n] = impseq(n0,n1,n2)
% Generates x(n) = delta(n-n0); n1 <= n <= n2
% ---------------------------------------------------------
% [x,n] = impseq(n0,n1,n2)
%
n = [n1:n2];
x = [(n-n0) == 0];
end
```

Function 2 MATLAB Codes

```
function [x,n] = stepseq(n0,n1,n2)

% Generates x(n) = u(n-n0); n1 <= n <= n2

% [x,n] = stepseq(n0,n1,n2)

n = n1:n2;              % Range of n

x = (n >= n0);          % 1 for n >= n0, else 0

end
```

Function 3 MATLAB Code

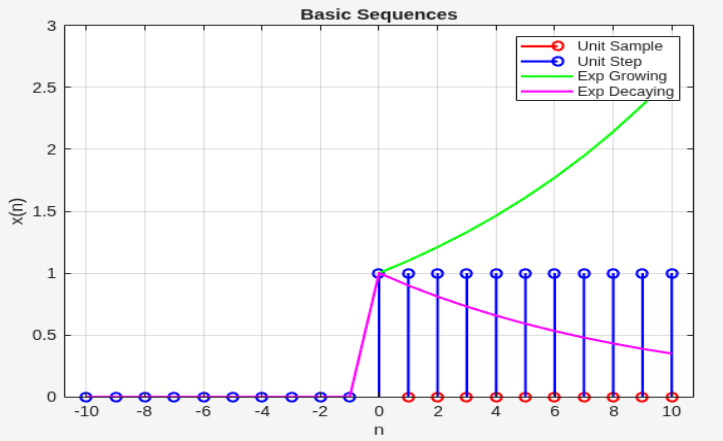
```
function [x,n] = expgseq(n0,n1,n2)
% Generates x(n) = exp(n-n0); n1 <= n <= n2
% [x,n] = expgseq(n0,n1,n2)

n = n1:n2;              % Range of n
x = exp(n-n0);          % Exponential growth
end
```

Function 4 MATLAB Code

```
function [x,n] = expdseq(n0,n1,n2)
% Generates x(n) = exp(-(n-n0)); n1 <= n <= n2
%
% [x,n] = expdseq(n0,n1,n2)

n = n1:n2;              % Range of n
x = exp(-(n-n0));       % Exponential decay
end
```

Write MATLAB code to call all the functions and plot it on one figure with proper labelling

```
% Parameters
n0 = 0;     n1 = -10;     n2 = 10;
% Generate sequences
[x1,n] = impseq(n0,n1,n2);     % Unit Sample
[x2,~] = stepseq(n0,n1,n2);    % Unit Step
[x3,~] = expgseq(n0,n1,n2);    % Exponential Growing
[x4,~] = expdseq(n0,n1,n2);    % Exponential Decaying
% Plotting
figure;
stem(n,x1,'r','LineWidth',1.5); hold on;
stem(n,x2,'b','LineWidth',1.5);
plot(n,x3,'g','LineWidth',1.5);
plot(n,x4,'m','LineWidth',1.5);
```



Lab Task 2: Recursive Function

Write a MATLAB recursive function code to compute factorial of number. Also verify your function.

Solution

```
% Define recursive function
function f = myFactorial(n)
   if n == 0 || n == 1
      f = 1;              % Base case
   else
      f = n * myFactorial(n-1);  % Recursive step
   end
end
```

```
% Ask user for input
num = input('Enter a number to calculate factorial: ');

% Call recursive function
result = myFactorial(num);

% Display results
disp(['Factorial of ', num2str(num), ' is: ', num2str(result)]);
disp(['MATLAB factorial(', num2str(num), ') = ', num2str(factorial(num))]);
```

```
y: /home/muhammadahmadhabib
Command Window
Enter a number to calculate factorial: 7
Factorial of 7 is: 5040
MATLAB factorial(7) = 5040
>>
```

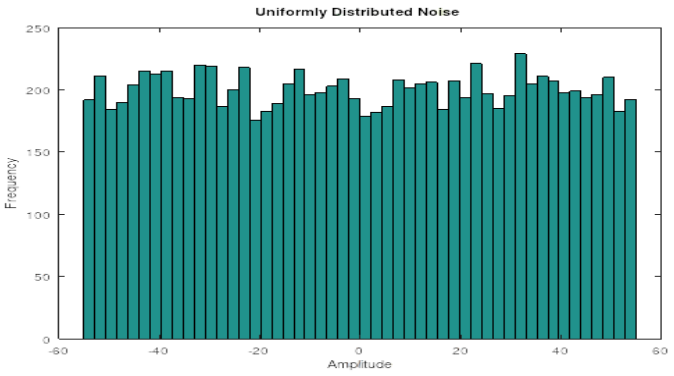
Lab Task 3: Random Number Generation

In practice, random number generators are often used to simulate the effect of noise like signals and other random phenomenon encountered in the physical world. Such noise is present in electronic devices and systems and usually limits our ability to communicate over large distance. By generating such noise on computer, we can study its effect through simulation.

Write a MATLAB code to generate uniformly distributed noise in an interval [-R, R], where R is your CIIT Registration number.

Solution

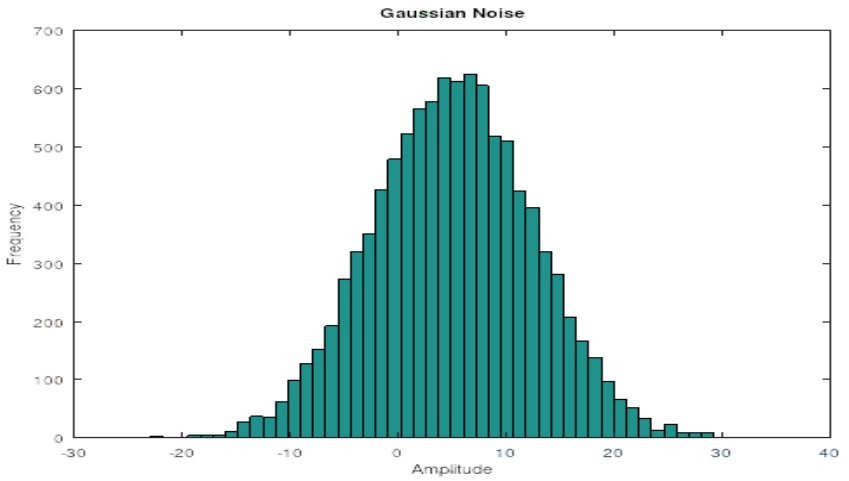
```
clc; clar all; close all;
A = -055;  %Roll Number
b = 055;
N = 10000;
noise_uniform = a + (b-a) * rand(N ,1);
figure;
histogram(noise_uniform, 50);
xlabel('Amplitude');
ylabel('Frequency');
title('Uniformly Distributed Nwoise in [-055, 055]');
```



Write a MATLAB code to generate a noise signal from gauss distribution of mean=B.M and variance=R, where B.M is your birth month and R is your CIIT Registration number.

Solution

```
% ----- Gaussian Noise -----
clc; clear all; close all;
R = 055;  % Roll No.
B = 5; % mean
N = 10000
variance = R;
std_dev = sqrt(variance);
noise_gaussian = std_dev
   * randn(N,1) + B;
figure;
hist(noise_gaussian, 50);
xlabel('Amplitude');
ylabel('Frequency');
title('Gaussian Noise');
```



Rubric for Lab Assessment

The student performance for the assigned task during the lab session was:			
Excellent	The student completed assigned tasks without any help from the instructor and showed the results appropriately.	4	
Good	The student completed assigned tasks with minimal help from the instructor and showed the results appropriately.	3	
Average	The student could not complete all assigned tasks and showed partial results.	2	
Worst	The student did not complete assigned tasks.	1	

Instructor Signature: _____ Date: _____

LAB # 2
To show Analog-to-Digital conversion experiment for discrete time signal operations using MATLAB

Objectives

- To explain conversion of continuous time signal to discrete time signal using MATLAB
- To explain quantization of continuous amplitude signal using MATLAB

Pre-Lab

To process analog signals by digital means, it is first necessary to convert them into digital form. The procedure is called *analog-to-digital* (*A/D*) conversion. A/D conversion is viewed as three step processes shown in Figure 1.

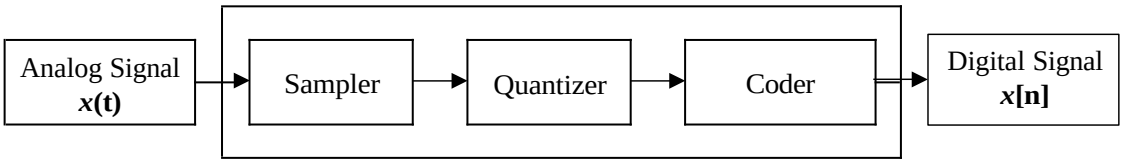


Figure 2.1: Analog to Digital Converter

Here we first develop concepts of sampling processing in the time domain. Figure 2.2shows an analog (continuous-time) signal (solid line) defined at every point over the time axis (horizontal line) and amplitude axis (vertical line). Hence, the analog signal contains an infinite number of points. It is impossible to digitize an infinite number of points. The infinite points cannot be processed by the digital signal (DS) processor or computer, since they require an infinite amount of memory and infinite amount of processing power for computations. Sampling can solve such a problem by taking samples at a fixed time interval as shown in Figure
3.2and Figure 3.3, where the time T represents the sampling interval or sampling period in seconds.

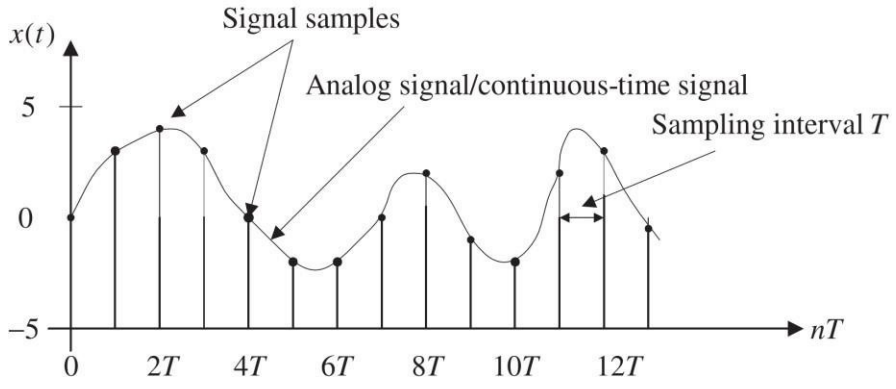


Figure 2.2 : Analog Signals and Discrete Time samples

For a given sampling interval T, which is defined as the time span between two sample points, the sampling rate is therefore given by

$$f_s = \frac{1}{T} \ \ samples\ per\ second\eth\ (Hz)$$

For example, if a sampling period is T = 125 microseconds, the sampling rate is $fs = \frac{1}{125}\mu s = 8000\frac{samples}{sec}$. After obtaining the sampled signal whose amplitude values are taken at the sampling instants, the processor is able to process the sample points. Next, we have to ensure that samples are collected at a rate high enough that the original analog signal can be reconstructed or recovered later. In other words, we are looking for a minimum sampling rate to acquire a complete reconstruction of the analog signal from its sampled version. If an analog signal is not appropriately sampled, aliasing will occur, which causes unwanted signals in the desired frequency band.

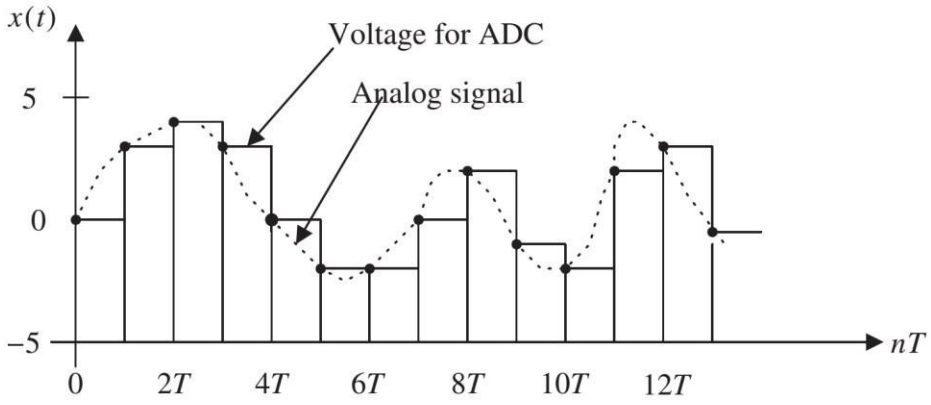


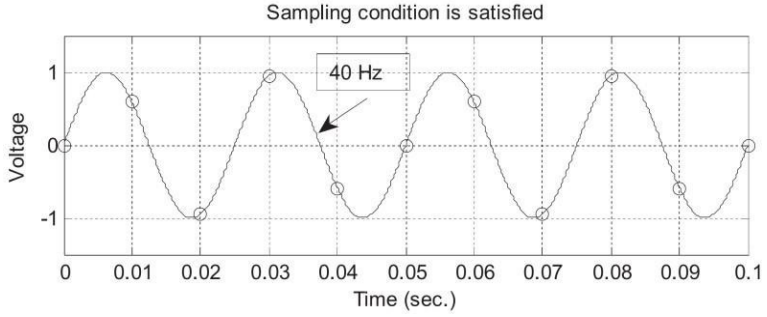
Figure 2.3: Quantized Discrete Time Signal

The sampling theorem guarantees that an analog signal can be in theory perfectly recovered as long as the sampling rate is at least twice as large as the highest-frequency component of the analog signal to be sampled. The condition is described as

$$f_s \geq 2f_{max}$$

Where f_{max} is the maximum-frequency component of the analog signal to be sampled. For example, to sample a speech signal containing frequencies up to 4 kHz, the minimum sampling rate is chosen to be at least 8 kHz, or 8,000 samples per second; to sample an audio signal possessing frequencies up to20 kHz, at least 40,000 samples per second, or 40 kHz, of the audio signal are required.

Figure 2.4illustrates sampling of two sinusoids, where the sampling interval between sample points is T=0.01 second, and the sampling rate is thus f_s= 100 Hz. The first plot in the figure displays a sine wave with a frequency of 40 Hz and its sampled amplitudes. The sampling theorem condition is satisfied since $2f_{max}$= 80 <f_s. The samples amplitude are labelled using the circles shown in the first plot.



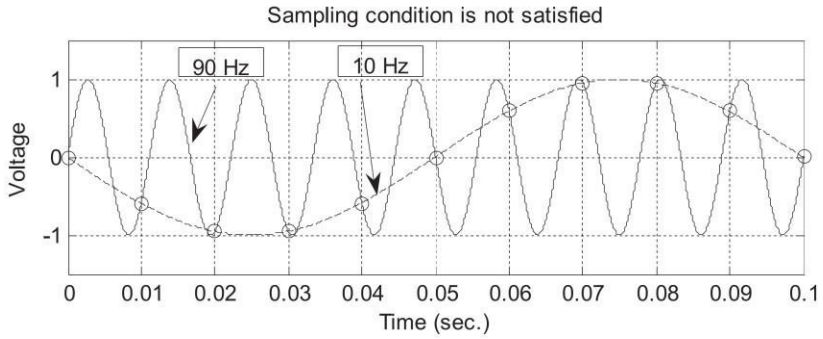


Figure 2.4: Plots of the appropriately sampled signals and non-appropriately sampled (aliased) signals.

We notice that the 40-Hz signal is adequately sampled, since the sampled values clearly come from the analog version of the 40-Hz sine wave. However, as shown in the second plot, the sine wave with a frequency of 90 Hz is sampled at 100 Hz. Since the sampling rate of 100 Hz is relatively low compared with the 90-Hz sine wave, the signal is under sampled due to $2f_{max}$= 180 >f_s . Hence, the condition of the sampling theorem is not satisfied. Based on the sample amplitudes labelled with the circles in the second plot, we cannot tell whether the sampled signal comes from sampling a 90-Hz sine wave (plotted using the solid line) or from sampling a 10-Hz sine wave (plotted using the dot-dash line). They are not distinguishable. Thus, they are aliases of each other. We call the 10-Hz sine wave the aliasing noise in this case, since the sampled amplitudes actually come from sampling the 90-Hz sine wave.

The process of converting analog voltage with infinite precision to finite precision is called the quantization process. For example, if the digital processor has only a 3-bit word, the amplitudes can be converted into eight different levels. A unipolar quantizer deals with analog signals ranging from 0 volt to a positive reference voltage, and a bipolar quantizer deals with analog signals ranging from a negative reference to a positive reference. The notations and general rules for quantization are as follows:

$$\Delta = \frac{x_{max} - x_{min}}{L}$$

$$L = 2^m$$

$$i = round\frac{x - x_{\min}}{\Delta}$$

Where $xmax$ and $xmin$ are the maximum value and minimum values, respectively, of the analog input signal x. The symbol L denotes the number of quantization levels, which is determined by above equation. Where is the number of bits used in ADC. The symbol Δ the step size of the quantizer or the ADC resolution.

Lab Tasks

Lab Task 1 a)

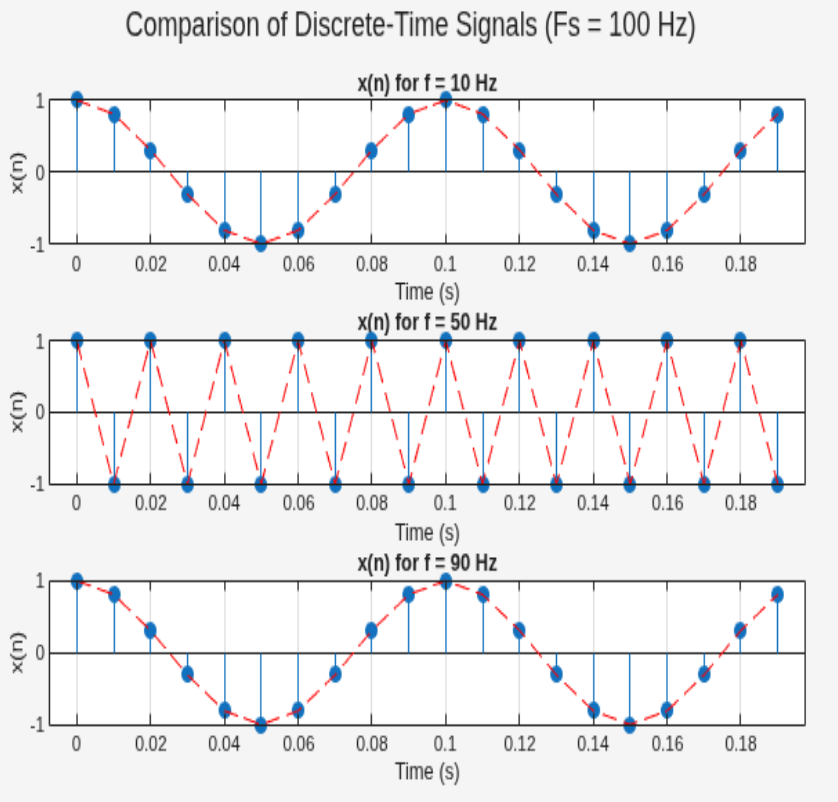
Consider the following continuous-time sinusoidal signal

$$x_a(t) = \cos(2\pi ft)\,, \qquad 0 \le t \le 2$$

Plot the discrete time signal x(n), 0 ≤ n ≤ 19 for sampling frequency F_S = 100 Hz and f = 10, 50 and 90 (Hz). Compare the graphs by identifying similarities and differences.

Solution

```
% Parameters
Fs = 100;        % Sampling frequency (Hz)
ts = 1/Fs;       % Sampling time
n = 0:19;        % Discrete time index
t = n * ts       % Time vector for samples

% Frequencies
f1 = 10;
f2 = 50;
f3 = 90;

% Signals
x1 = cos(2*pi*f1*t);
x2 = cos(2*pi*f2*t);
x3 = cos(2*pi*f3*t);

stem(t, x1, 'filled'); hold on;
plot(t, x1, 'r--');   % Connect points

stem(t, x2, 'filled'); hold on;
plot(t, x2, 'r—');

stem(t, x3, 'filled'); hold on;
plot(t, x3, 'r--');
```

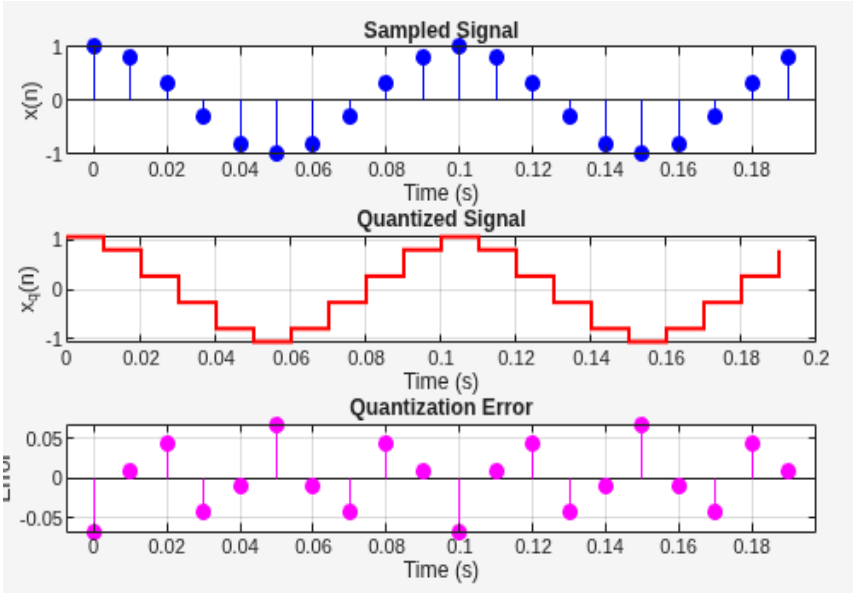


Lab Task 1 b)

Quantize signal x(n) (for *f* = 10Hz) using 4 bits and plot sampled, quantized and error signal on same figure.

Solution

```
% Parameters
Fs = 100;       % Sampling frequency (Hz)
ts = 1/Fs;      % Sampling time
n = 0:19;       % Discrete time index
t = n * ts;     % Time vector for samples
% Frequencies
f1 = 10;  f2 = 50;  f3 = 90;
% Signals
x1 = cos(2*pi*f1*t);
x2 = cos(2*pi*f2*t);
x3 = cos(2*pi*f3*t);
% Plotting
stem(t, x1, 'filled'); hold on;
plot(t, x1, 'r—');    % Connect points
stem(t, x2, 'filled'); hold on;
plot(t, x2, 'r—');
stem(t, x3, 'filled'); hold on;
plot(t, x3, 'r--');
```

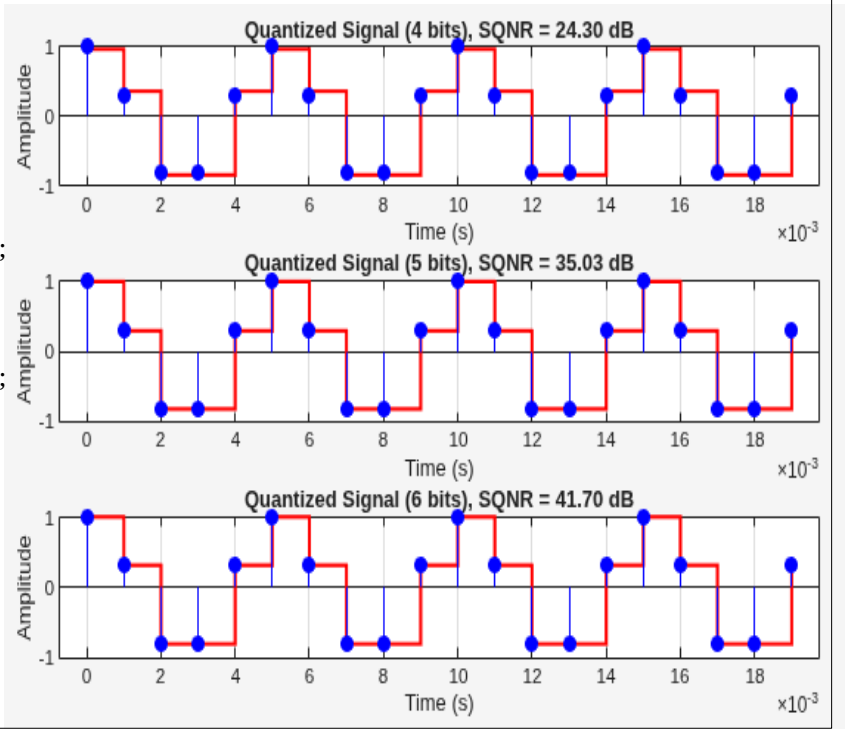


Lab Task 1 c)

Quantize the sampled signal x(n) (for *f* = 100Hz) using 4, 5 and 6 bits respectively. Comment the difference in output by computing SQNR. (F_S=200 Hz)

Solution

```
% Parameters
f = 200;                % Signal frequency
Fs = 1000;              % Sampling frequency
ts = 1/Fs;              % Sampling interval
n = 0:99;               % Samples
t = n*ts;               % Time vector
x = cos(2*pi*f*t);      % Original signal
% --- 4 bits ---
M4 = 4;   L4 = 2^m4;   D4 = (max(x)-min(x))/(L4-1);
xq4 = quant(x, D4);   err4 = x - xq4;
sqnr4 = 10*log10(sum(x.^2)/sum(err4.^2));
% --- 5 bits ---
M5 = 5;   L5 = 2^m5;   D5 = (max(x)-min(x))/(L5-1);
xq5 = quant(x, D5);    rr5 = x - xq5;
sqnr5 = 10*log10(sum(x.^2)/sum(err5.^2));

% --- 6 bits ---
m6 = 6;
L6 = 2^m6;
D6 = (max(x)-min(x))/(L6-1);
xq6 = quant(x, D6);
err6 = x - xq6;
sqnr6 = 10*log10(sum(x.^2)/sum(err6.^2));
```

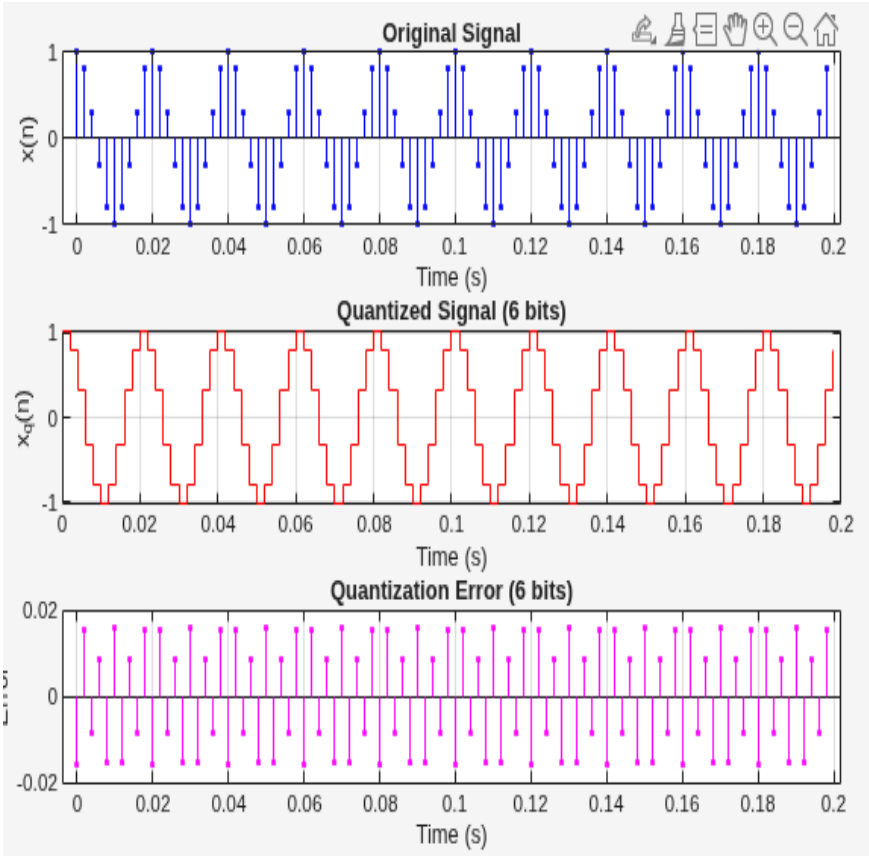


Lab Task 2

Write your own custom function which is able to simulate a uniform quantizer. The inputs should contain sampled signal, number of bits used in quantization and outputs will be quantized signal,error signal and SQNR.

Solution

```
Parameters
f = 50;                 % Signal frequency
Fs = 500;               % Sampling frequency
ts = 1/Fs;
n = 0:99;
t = n*ts;
x = cos(2*pi*f*t);      % Input signal

--- Custom Quantizer Function Calls ---
[xq4, err4, sqnr4] = myQuantizer(x, 4);
[xq5, err5, sqnr5] = myQuantizer(x, 5);
[xq6, err6, sqnr6] = myQuantizer(x, 6);

Display SQNR results
disp('Custom Quantizer SQNR Results (dB):');
disp(['4 bits = ', num2str(sqnr4)]);
disp(['5 bits = ', num2str(sqnr5)]);
disp(['6 bits = ', num2str(sqnr6)]);

Plot Example (6-bit)
subplot(3,1,1);     stem(t, x, 'b.');      grid on;
title('Original Signal'); xlabel('Time (s)');

subplot(3,1,2);     stairs(t, xq6, 'r');   grid on;
title('Quantized Signal (6 bits)');

subplot(3,1,3);     stem(t, err6, 'm.');   grid on;
title('Quantization Error (6 bits)'); ;
```



Rubric for Lab Assessment

The student performance for the assigned task during the lab session was:			
Excellent	The student completed assigned tasks without any help from the instructor and showed the results appropriately.	4	
Good	The student completed assigned tasks with minimal help from the instructor and showed the results appropriately.	3	
Average	The student could not complete all assigned tasks and showed partial results.	2	
Worst	The student did not complete assigned tasks.	1	

Instructor Signature: _____ Date: _____

LAB # 3
To show the response of the Linear Time Invariant system for arbitrary discrete time inputs using MATLAB

Objectives

- To explain discrete time convolution sum using MATLAB
- To verify the properties of convolution using MATLAB

Pre-Lab

Discrete Time LTI System and Convolution Sum

Discrete Time Systems

The function of a discrete time system is to process a given ***input sequence*** to generate an ***output sequence***. In the most applications, the discrete time system used is single input and single output system, as shown in the diagram below.

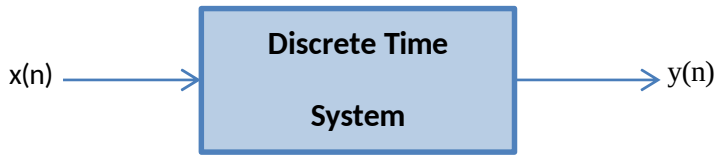


Figure 1: Schematic representation of discrete time systems

Mathematically, a discrete time system is described as an operator ***T{.}*** that takes a sequence x(n) and transforms it into another sequence y(n). That is

$$y(n) = T[x(n)]$$

In practical discrete time system, all the signals are digital signals and operations on such signals also leads to the digital signals. Such a discrete time is usually called "***Digital Filter***". We shall refer to discrete time system as a digital filter whether or not it has been implemented using finite precision arithmetic.

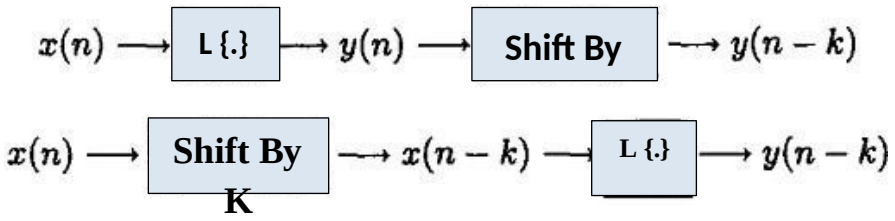
Linear System

A discrete time system is a linear if and only if it satisfies the principle of superposition

$$L[a_1x_1(n) + a_2x_2(n)] = a_1L[x_1(n)] + a_2L[x_2(n)], \forall a_1, a_2, x_1(n), x_2(n)$$

Linear Time Invariant System

A linear system in which an input-output pair ***x(n)*** and ***y(n)*** is invariant to a shift n in time is called a linear time invariant system. For an LTI system the liner operator and shift operators are reversible. The impulse response of the LTI system is given by ***h(n)***. Mathematical operation shown in the below equation is called linear convolution sum.



$$y[n] = LTI\,[x(n)] = \sum_{k=-\infty}^{\infty} x(k)h(n-k)$$

Simple Discrete Time System

A simple example of discrete time system is the accumulator defined by the input relationship.

$$y[n] = \sum_{l=-\infty}^{n} x(l)$$

It accumulates all input sample values from $-\infty$ *to*n.

Pre-Lab Tasks

(Attach extra A4 White page if needed)

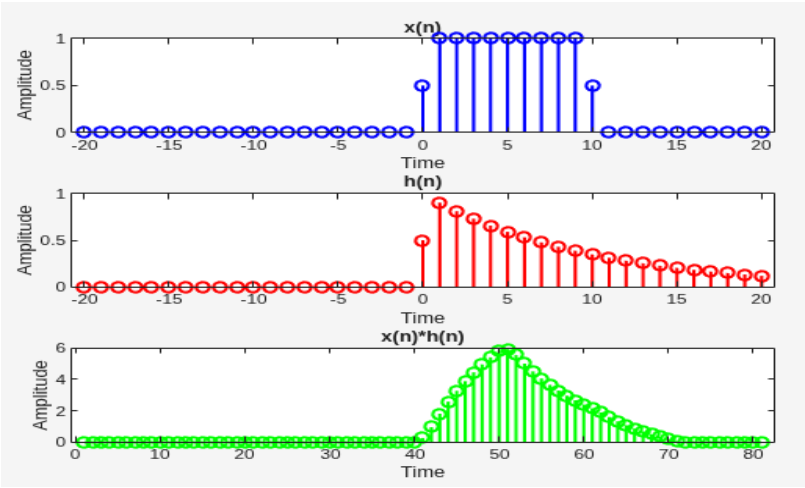
Let the following rectangular pulse **x(*n*)** be an input to an LTI system with impulse response $h(n)$, determine the output ***y*(*n*)**. Plot the input signal and output signal.

$$x(n) = u(n) - u(n-10)$$

$$h(n) = (0.9)^n u(n)$$

=

N = -20:20;
x = heaviside(n) – heaviside(n-10);
h = 0.9 .^ n .* heaviside(n);
c = conv (x , h);
Subplot (311);
stem (n , x , 'b', 'LineWidth' , 1.5)
subplot (312);
stem (n,h,'r', 'LineWidth' , 1.5)
Subplot (313) ;
stem (c , 'g', 'LineWidth' , 1.5)



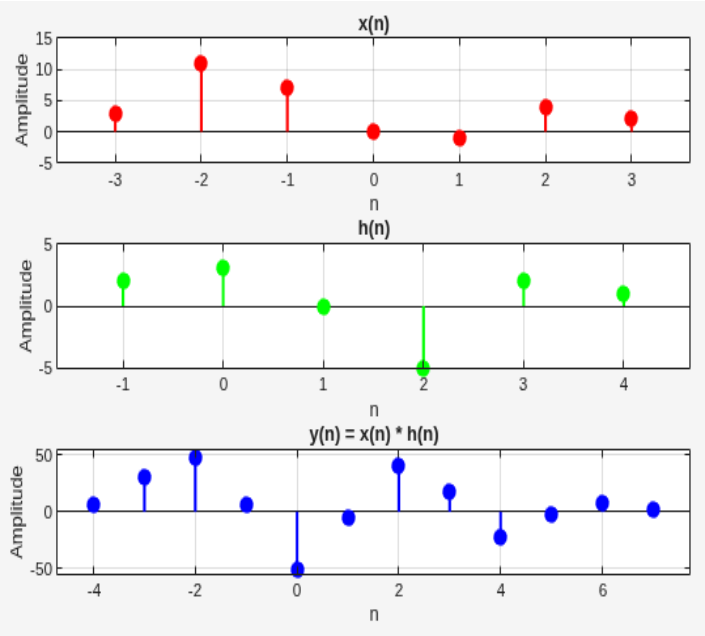
Lab Tasks

Lab Task 1

Given the following two sequences, determine and plot the convolution result y(n),also plot x(n) and h(n) .

$$x(n) = \left[3, \underset{\uparrow}{11}, 7, 0, -1, 4, 2\right], \quad -3 \le n \le 3; \qquad h(n) = \left[2, \underset{\uparrow}{3}, 0, -5, 2, 1\right], \quad -1 \le n \le 4$$

```
n1 = -3 : 3;
x = [3, 11, 7, 0, -1, 4, 2];

n2 = -1 : 4;
h = [2, 3, 0, -5, 2, 1];

n = -4 : 7;
y = conv(x, h);

Subplot(3,1,1);    stem(n1, x, 'r', 'filled')
xlabel('n');   ylabel('Amplitude');   title('x(n)');
ylim([-5 15]);   grid on;

Subplot(3,1,2);    stem(n2, h, 'g', 'filled')
xlabel('n'); ylabel('Amplitude');   title('h(n)');
Ylim([-5 5]);    grid on;

Subplot(3,1,3);    stem(n, y, 'b', 'filled')
xlabel('n'); ylabel('Amplitude');   title('y(n) = x(n) * h(n)');
ylim([-55 55]);   grid on;
```

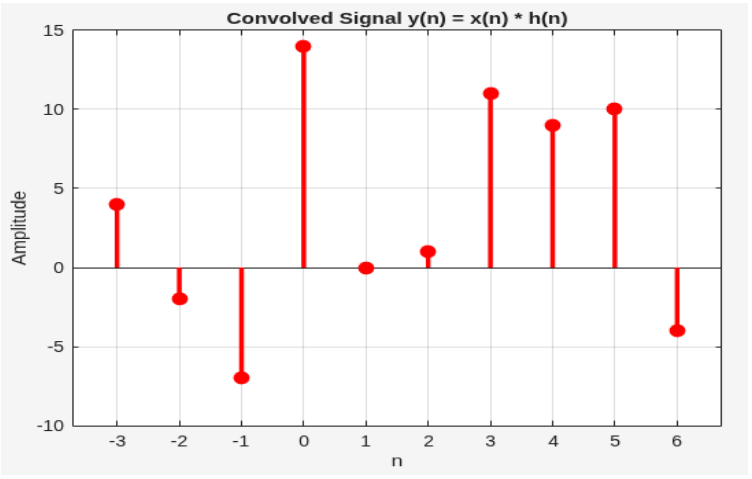


Lab Task 2

Write a MATLAB function to systematically develop the sequence y(n)generated by the convolution of the two- finite-length sequences x(n)and h(n).Program should call for the input sequences and their indices vectors.

```
Input signal x
x = input('Enter array for input signal x:');
Time index range for x
n1 = input('Specify starting value for x: ');
n2 = input('Specify ending value for x: ');
nx = n1:n2;
Impulse response h
h = input('Enter array for impulse response h:');
Time index range for h
n3 = input('Specify starting value for h: ');
n4 = input('Specify ending value for h: ');
nh = n3:n4;
[y, n] = convolution(x, nx, h, nh);
Plot the convolved signal
stem(n, y, 'filled');
xlabel('n');    ylabel('Amplitude');
title('Convolved Signal y(n) = x(n) * h(n)');


 --- Function ---
function [y ,n] = convolution(x , n1, h , n2)
n = (n1(1) + n2(1)) : (n1(end) + n2(end));
y = conv(x ,h)
end
```

```
Enter array for input signal x:[4 -2 5 0 3 -1]
Specify starting value for x: -2
Specify ending value for x: 3
Enter array for impulse response h:[1 0 -3 2 4]
Specify starting value for h: -1
Specify ending value for h: 3

y =        4   -2   -7   14   0   1   11   9   10   -4 >>
```



Lab Task 3

Show that convolution of a length -M sequence with a length-N sequence leads to a sequence of length (M+N-1).

```
n1 = -3:3;
n2 = -1:4;

X = [4 -2 5 0 3 -1];
H = [1 0 -3 2 4];

length_xh = length(x) + length(h) - 1;
   y = conv(x,h);
length_conv = length(y);

  if (length_xh == length_conv)
     disp('Length(y) = M + N - 1 ');
  else
     disp('Length(y) != M + N - 1 ');
end
```

```
N1 =    -2 -1 0 1 2 3

N2 =    -1 0 1 2 3

X =     4 -2 5 0 3 -1

H =     1 0 -3 2 4

length_xh =  10

Y =   4 -2 -7 14 0 1 11 9 10 -4

length_conv = 10

Length(y) = M + N - 1    >>
```

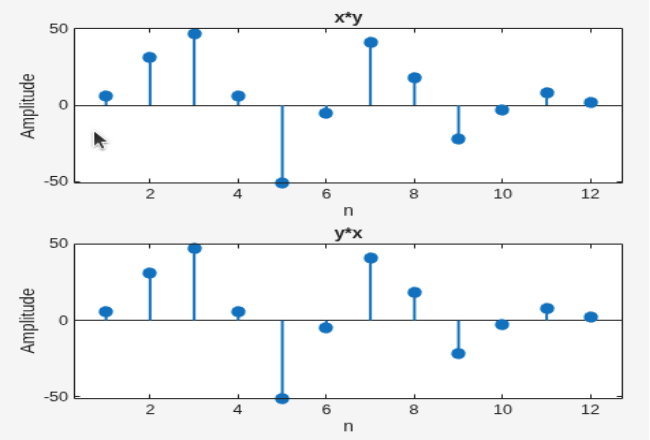
Lab Task 4

Verify following properties of Convolution:

1. Commutative Law
2. Associative Law
3. Distributive Law

```
X = [3,11,7,0,-1,4,2];  y = [2,3,0,-5,2,1];  z = [2,3,0,-5,2,1];
r1 = conv(x,y);   r3 = conv(y,z);

lhs_assoc = conv(r1,z);      rhs_assoc = conv(x,r3);   figure;
Subplot(2,1,1);      stem(lhs_assoc,'filled','linewidth',1.5);
Subplot(2,1,2);      stem(rhs_assoc,'filled','linewidth',1.5);
```

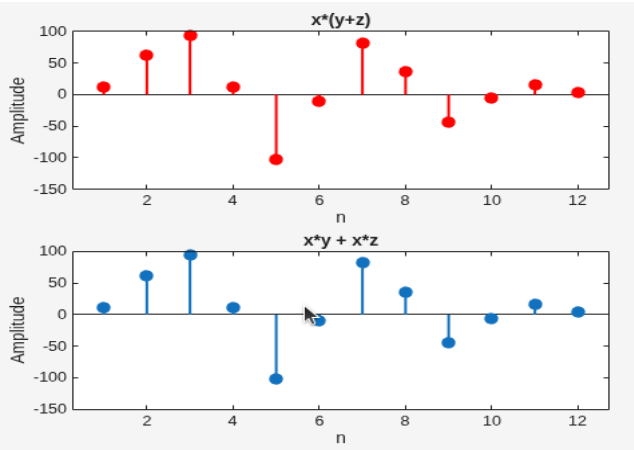
Associative Property

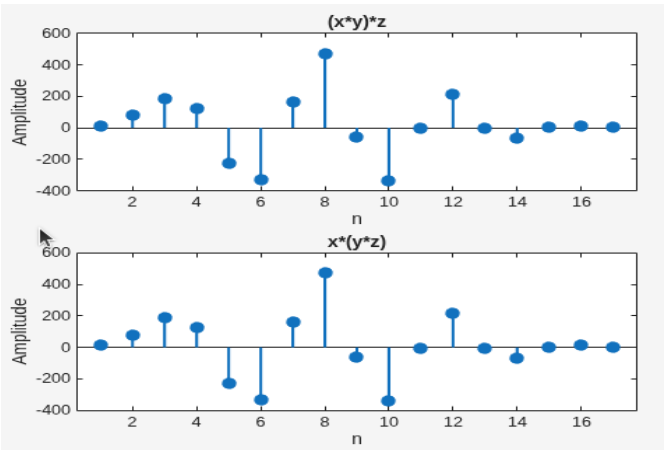



```
X = [3,11,7,0,-1,4,2];  y = [2,3,0,-5,2,1];  z = [2,3,0,-5,2,1];

c1 = conv(x,y);        c2 = conv(y,x);   figure;
Subplot(2,1,1);      stem(c1,'filled','linewidth',1.5);
Subplot(2,1,2);      stem(c2,'filled','linewidth',1.5);
```

```
X = [3,11,7,0,-1,4,2];   y = [2,3,0,-5,2,1];  z = [2,3,0,-5,2,1];

lhs_dist = conv(x, y+z);      rhs_dist = conv(x,y) + conv(x,z);
Subplot(2,1,1);      stem(lhs_dist,'r','filled','linewidth',1.5);
Subplot(2,1,2);      stem(rhs_dist,'filled','linewidth',1.5);
```

Commutative Property





Rubric for Lab Assessment

The student performance for the assigned task during the lab session was:			
Excellent	The student completed assigned tasks without any help from the instructor and showed the results appropriately.	4	
Good	The student completed assigned tasks with minimal help from the instructor and showed the results appropriately.	3	
Average	The student could not complete all assigned tasks and showed partial results.	2	
Worst	The student did not complete assigned tasks.	1	

Instructor Signature: _____ Date: _____

LAB # 4
To explain the z-Transform, properties of z-Transform and to analyze the Linear Time Invariant system in z Domain using MATLAB

Objectives

- To study the signals and systems in Z Domain.

Pre-Lab

Z-Transform

In this Lab we introduce the z-transform, z-transform is counterpart of Laplace transform when dealing with Discrete-Time signals. It is employed to transform difference equations that describe the input /output relationships of discrete time systems into algebraic equations and is very useful tool for the analysis and design of discrete-time systems.

Mathematical Definition

As Laplace transformation is a more general transform as compared to Fourier Transform for continuous-time signals, z-Transform is a more general transform than discrete-time Fourier Transform when dealing with discrete-time signal. A discrete-time signal is defined in the discrete time domain n; that is, it is given by a function f[n], $n \in Z$. z-Transform is denoted by the symbol $Z\{.\}$ and expresses a signal in z-domain, i.e., the signal is given by a function F(z). The mathematical expression is

$$F(z) = Z\{f[n]\}$$

In other words, the z-transform of a function f [n] is F(z). The mathematical expression of the two-sided (orbilateral) z-transform is

$$F(z) = Z\{f[n]\} = \sum_{n=-\infty}^{\infty} f[n]z^{-n},$$

Where z is a complex variable. Setting the lower limit of the sum from minus infinity to zero yields the onesided (or unilateral) z-transform whose mathematical expression is

$$f(z) = Z\{f[n]\} = \sum_{n=0}^{\infty} f[n]z^{-n}.$$

In order to return from the z-domain back to the discrete-time domain, the inverse z-transform is applied. The inverse z-transform is denoted by the symbol $Z - 1\{.\}$; that is, one can write

$$\text{f}[n] = Z^{-1}\{\text{F(z)}\}.$$

The mathematical expression of the inverse z-transform is

$$x[n] = \frac{1}{2\pi j}\oint X(z)z^{n-1}\,dz.$$

MATLAB Commands Related with Z Transform

[H , ω] = **freqz** (B, A, N);

[H,T] = **impz**(B,A,N)

zplane(z,p);

[z, P, k] = **tf2zp**(B,A);

[B,A] = **zp2tf**(z,P,k);

[R,P,K]= **residuez**(B,A);

Pre Lab Tasks

(Attach extra A4 White page if needed) A z- Transform H(z) is given below.

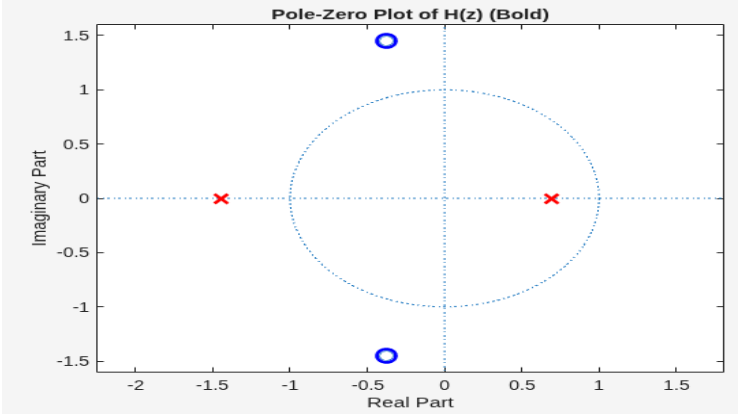
$$H(z) = \frac{4z^2 + 3z + 9}{4z^2 + 3z - 4}$$

Write a MATLAB program that take two vectors 'n' & 'd' that are the numerator and denominator coefficients. Program should find the following.

a) Pole and Zero plot

b) Partial fraction expansion of the z transform

```
N = [4 3 9];
d = [4 3 -4];% Zero-Pole-Gain form
[z, p, k] = tf2zp(n, d);% Bold pole-zero
 plotzplane(z, p);
title('Pole-Zero Plot of H(z) (Bold)');
grid on;
% Partial fraction expansions
[R, P, K] = residuez(n, d);
disp('--- Partial Fraction Expansion ---');
disp('Residues (R):');      disp(R);
disp('Poles (P):');         disp(P);
disp('Direct Term (K):');     disp(K);
```




```
--- Partial Fraction Expansion ---
Residues (R):    1.0544      2.1956
Poles (P):      -1.4430      0.6930
Direct Term (K):       -2.2500  >>
```

Lab Tasks

Lab Task 1

Determine the rational Z-Transform from its poles and zero locations. The zeros are at

$\zeta_1 = 0.21,\ \zeta_2 = 3.14\ ,\ \zeta_3 = -0.3 + j\,0.5\ ,\ \zeta_4 = -0.3 - j0.5$

And poles are at

$\lambda_1 = -0.45\ ,_2 = -0.67,\ \lambda_3 = 0.81 + j\,0.72\ ,\ \lambda_4 = 0.81 - j0.72$ and the gain constant is 2.2.

Define zeros, poles, and gain

```
z = [0.21; 3.14; -0.3 + 0.5j; -0.3 – 0.5j];
p = [-0.45; -0.67; 0.81 + 0.72j; 0.81 – 0.72j];
K = 2.2;
```

Convert zero-pole-gain

```
[b, a] = zp2tf(z, p, k)
H = tf(b, a, 1)     % Covert Transfer Function
```

```
Command Window
New to MATLAB? See resources for Getting Started.

b =
    2.2000   -6.0500   -2.2233   -1.6354    0.4932

a =
    1.0000   -0.5000   -0.3384    0.8270    0.3541

H =
  2.2 z^4 - 6.05 z^3 - 2.223 z^2 - 1.635 z + 0.4932
  ------------------------------------------------
    z^4 - 0.5 z^3 - 0.3384 z^2 + 0.827 z + 0.3541
```

Lab Task 2

Using MATLAB, determine the partial fraction expansion of the z transform G(z), given by

$$G(z) = \frac{18\,z^3}{18z^3 + 3\,z^2 - 4z - 1}$$

```
syms z
num = [18 0 0 0];
den = [18 3 -4 -1];
G = poly2sym(num,z) / poly2sym(den,z);
disp('G(z) =')
pretty(G)
disp('Partial Fraction Expansion:')
pretty(partfrac(G))
```

```
Command Window
New to MATLAB? See resources for Getting Started.

G(z) =
                3
            18 z
- ------------------------
         3      2
  - 18 z  - 3 z  + 4 z + 1

Partial Fraction Expansion:
      9              26              2
------------ - ------------ + ------------ + 1
(2 z - 1) 25   (3 z + 1) 25              2
                                5 (3 z + 1)

>>
```

Lab Task 3

Determine inverse z transform for the following cases without using iztrans command.

a) $Y_1(z) = \frac{z\,(z-1)}{(z+1)(z+\frac{1}{3})}$, $|z| > 1$

b) $Y_2(z) = \frac{z\,(z-1)}{(z+1)(z+\frac{1}{3})}$, $|z| < 1/3$

c) $Y_3(z) = \frac{z\,(z-1)}{(z+1)(z+\frac{1}{3})}$, $\frac{1}{3} < |z| < 1$

```
clc; clear; close all
syms z
num = [1 -1 0];
den = conv([1 1],[1 1/3]);
H = poly2sym(num,z) / poly2sym(den,z);
disp('H(z) ='), pretty(H)
disp('Partial fraction:')
pretty(partfrac(H))
```

```
Command Window
New to MATLAB? See resources for Gettin

H(z) =
         2
     - z  + z
- ------------
     2    4 z    1
    z   + --- + -
           3     3

Partial fraction:
    2         3
------- - ----- + 1
3 z + 1    z + 1

>>
```

Rubric for Lab Assessment

The student performance for the assigned task during the lab session was:			
Excellent	The student completed assigned tasks without any help from the instructor and showed the results appropriately.	4	
Good	The student completed assigned tasks with minimal help from the instructor and showed the results appropriately.	3	
Average	The student could not complete all assigned tasks and showed partial results.	2	
Worst	The student did not complete assigned tasks.	1	

Instructor Signature: _____ Date: _____

LAB # 5
To begin with the practical implementation of Digital Signal Processing concepts using hardware (DSP Starter Kits TMS320C6713)

Objectives

- To assemble first experiment on DSP Kit 6713 using Code Composer Studio.
- To identify and troubleshoot the problems in the hardware implementation of first experiment.

Pre-Lab

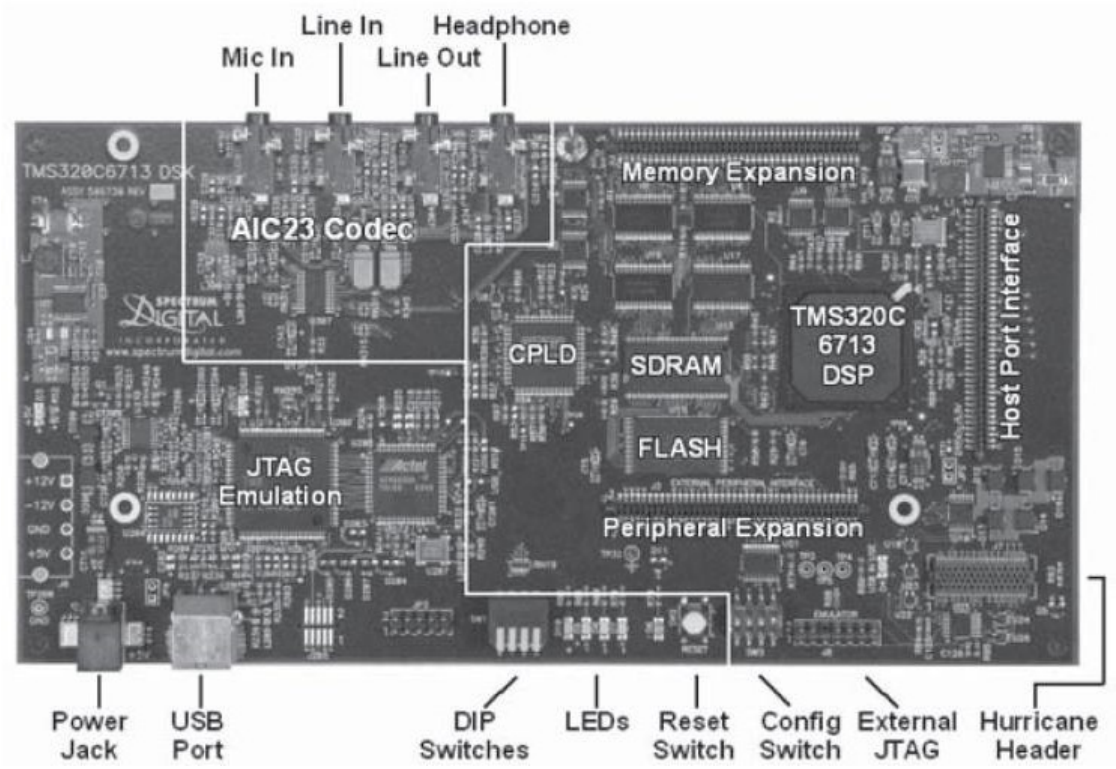
Digital signal processing is one of the core technologies, in rapidly growing application areas, such as wireless communications, audio and video processing and industrial control. The number and variety of products that include some form of digital signal processing has grown dramatically over the last few years. DSP has become a key component, in many of the consumer, communications, medical and industrial products which implement the signal processing using microprocessors, Field Programmable Gate Arrays (FPGAs), Custom ICs etc. Due to increasing popularity of the above-mentioned applications, the variety of the DSP-capable processors has expanded greatly. DSPs are processors or microcomputers whose hardware, software, and instruction sets are optimized for high-speed numeric processing applications, an essential for processing digital data, representing analog signals in real time. The DSP processors have gained increased popularity because of the various advantages like reprogram ability in the field, cost-effectiveness, speed, energy efficiency etc.

The hardware experiments in the DSP lab are carried out on the Texas Instruments TMS320C6713 DSP Starter Kit (DSK), based on the TMS320C6713 floating point DSP running at 225 MHz. The basic clock cycle instruction time is 1/(225 MHz)= 4.44 nanoseconds. During each clock cycle, up to eight instructions can be carried out in parallel, achieving up to 8×225 = 1800 million instructions per second (MIPS).

Features of DSK6713

Key features of TMS320C6713 DSP Starter Kit (DSK6713) are:

- Operate at 225 MHz
- AIC23 stereo codec
- 16 MB of synchronous DRAM
- 512 KB of non-volatile Flash memory
- 4 user accessible LEDs and DIP switches
- Software board configuration through registers implemented in CPLD
- Configurable boot options
- Standard expansion connectors for daughter card use
- JTAG emulation through on-board JTAG emulator with USB host
- Interface or external emulator
- Single voltage power supply (+5V)



Complex Programmable Logic Device (CPLD)

A **Complex Programmable Logic Device (CPLD)** is a combination of a AND/OR array. The AND/OR array can be reprogrammed to implement different logic functions. The CPLD logic is used to implement functionality specific to the DSK. The CPLD eliminate the need for additional discrete devices to be used with DSK. CPLDs provide following features:

- Reprogrammable Device
  - Lowers development cost
  - Saves time
  - High performance, low power operation
- Simple and easy to use
  - Fits easily into existing design flow
- Low cost
  - Reprogram ability: No need to purchase new equipment every time
  - Lowers design cost, system cost, and maintenance cost
- Nonvolatile
  - Save data even on power down
  - CPLD functions available instantly on system power up
- Security
  - Almost impossible to steal stored design

Peripherals

The TMS320C6713 devices contain peripherals for communication with off-chip memory, co-processors, host processors and serial devices. Peripherals of DSK6713 are described below.

Host Port Interface

The Host-Port Interface (HPI) is a 16-bit wide parallel port through which a host processor can directly access the CPUs memory space. The host device functions as a master to the interface, which increases ease of access. The host and CPU can exchange information via internal or external memory. The host also has direct access to memory-mapped peripherals. The HPI is connected to the internal memory via a set of registers.

General Purpose Input/Output (GPIO)

This is a 16-bit port provide general purpose pins that can be configured as input or output. When configured as an output, data can be written to an internal register to control the state driven on the output pin. When configured as an input, the state of the input pin can be detected by reading the state of an internal register. In addition, the GPIO peripheral can produce CPU interrupts in different interrupt generation modes.

External Memory Interface

The External Memory Interface (EMIF) supports several external memory devices. This feature allows additional data and program memory space than that is available on-chip.

- The devices supported are:
- Synchronous burst SRAM (SBSRAM) and Synchronous DRAM (SDRAM)
- Asynchronous SRAM, ROM, and FIFOs
- External shared-memory devices like microcontrollers

Multichannel Buffered Serial Port

The multichannel buffered serial port (McBSP) is based on the standard serial port interface. It provides:

- Full-duplex communication
- Double-buffered data registers, which allow a continuous data stream
- Independent framing and clocking for reception and transmission
- Direct interface to industry-standard codecs, analog interface chips (AICs)
- Multichannel transmission and reception of up to 128 channels
- An element sizes of 8-, 12-, 16-, 20-, 24-, or 32-bit
- μ-Law and A-Law companding

Multichannel Audio Serial Port

The DSK6713 includes two Multichannel Audio Serial Ports (McASP). The McASP interface modules support one transmit and one receive clock zone. Each of the McASP has eight serial data pins which can be individually allocated to any of the two zones.

Audio Codec

"The DSK6713 uses a Texas Instruments AIC23 (part #TLV320AIC23) stereo codec for input and output of audio signals". Audio codec provides two input and two output ports for analog signal as shown in fig4.1. It provide two way communications as, it samples the audio signal from mic-in or line-in and convert back sample into analog signal.

Timers

The DSK6713 has two 32-bit general-purpose timers that can be used to:

- Time events
- Count events

- Generate pulses
- Interrupt the CPU
- Send synchronization events to the EDMA controller

Code Composer Studio (CCS)

CCS provides an IDE to incorporate the software tools. CCS includes tools for code generation, such as a C compiler, an assembler, and a linker. It has graphical capabilities and supports real time debugging. It provides an easy-to-use software tool to build and debug programs.
The C compiler compiles a C source program with extension .c to produce an assembly source file with extension.*asm*. The assembler assembles an *.asm* source file to produce a machine language object file with extension.*obj*. The linker combines object files and object libraries as input to produce an executable file with extension.*out*. This executable file can be loaded and run directly on the C6713 processor.
In the lab we will use Code Composer Studio v3.1 run on Win XP.

Types of Files

There are a number of files with different extensions useful in building project. These are discussed below:

- **file.pjt:** file needed to create and build project
- **file.c:** C source program created by user
- **file.asm:** Assembly source program created by either user or by the C compiler
- **file.h:** file needed to support different functions in C source program
- **file.lib:** library file, such as the DSK6713 board support library file dsk6713.lib
- **file.cmd:** linker command file that maps sections to memory
- **file.obj:** file created by the assembler
- **file.out:** file created by the linker to be loaded on the C6713 processor to run C source program functionality.
- **file.cdb:** configuration file when using DSP/BIOS (different for each family of kits)

Header Files

There are a number of header files that should be included in project. It contains some special function to be performed. Following are some header files discussed. These files are useful in working with DSP kits

- **dsk6713.h:** It contains the definitions of macros require for the functionality of DSP board. No project can be built without it.
- **dsk6713_led.h:** It contains the definitions of macros require for the LEDs on board.
- **dsk6713_dip.h:** It contains the definitions of macros require for the DIP switches available on board.
- **dsk6713_aic23.h:** It contains the definitions of macros require for the functionality of Audio Codec.
- **csl.h:** It contains the definitions of macros require for the Chip Support Library functions.
- **csl_mcbsp.h:** It contains the definitions of macros require for the operation of MCBSP.
- **csl_gpio.h:** It contains the definitions of macros require for the operation of GPIO
- **csl_irq.h:** It contains the definitions of Interrupt requests, mapping, reset, enable and disable it.

Lab Tasks

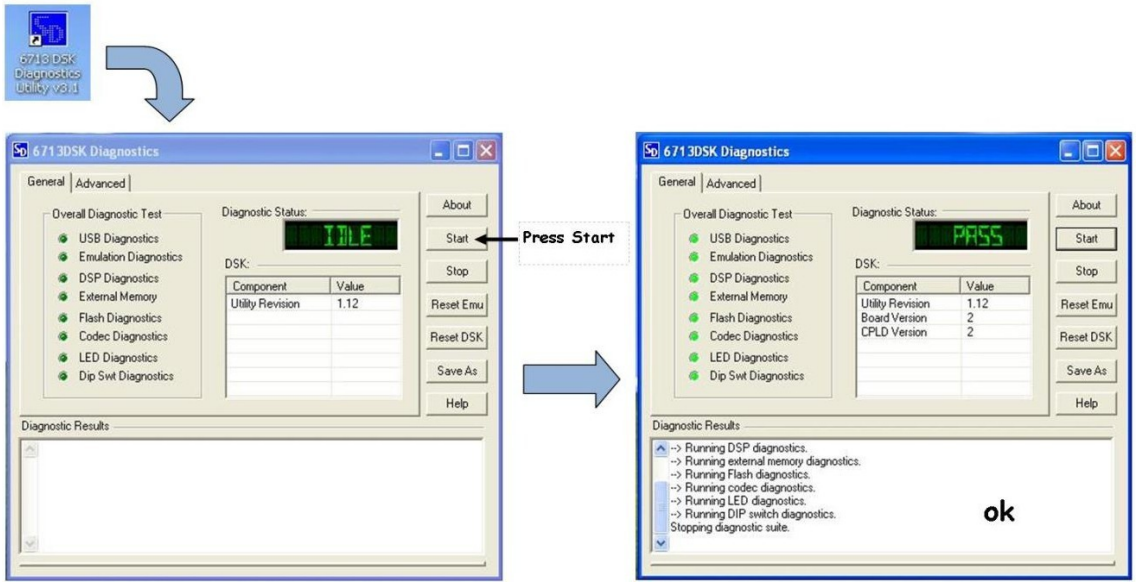
Task 1: Build your first project

Join DSP Kit to PC with USB JTAG interface. Power the kits with 5V standard adapter for DSK6713. Precaution: *DSK6713 are very sensitive to static charges. So don't touch kit after power up it might damage it.*

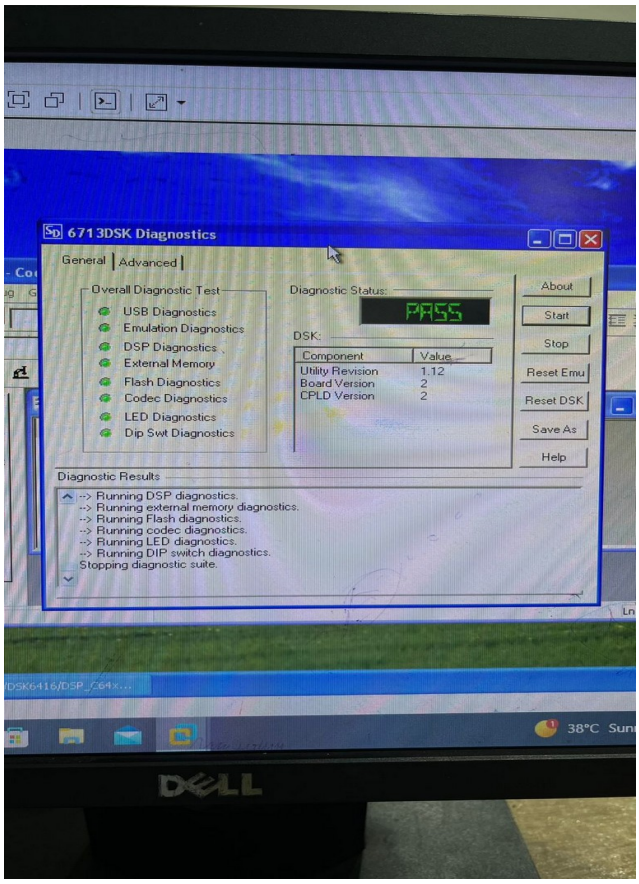
When DSK6713 is powered up it execute Power On Self-Test (POST) program stored in its flash. POST take 10-15 seconds to execute. POST checks all the subsystems of DSK6713. Output 1 kHz sinusoid to Audio Codec for one second. If POST is successful it will blink all 4-leds for three times and then stabilize.

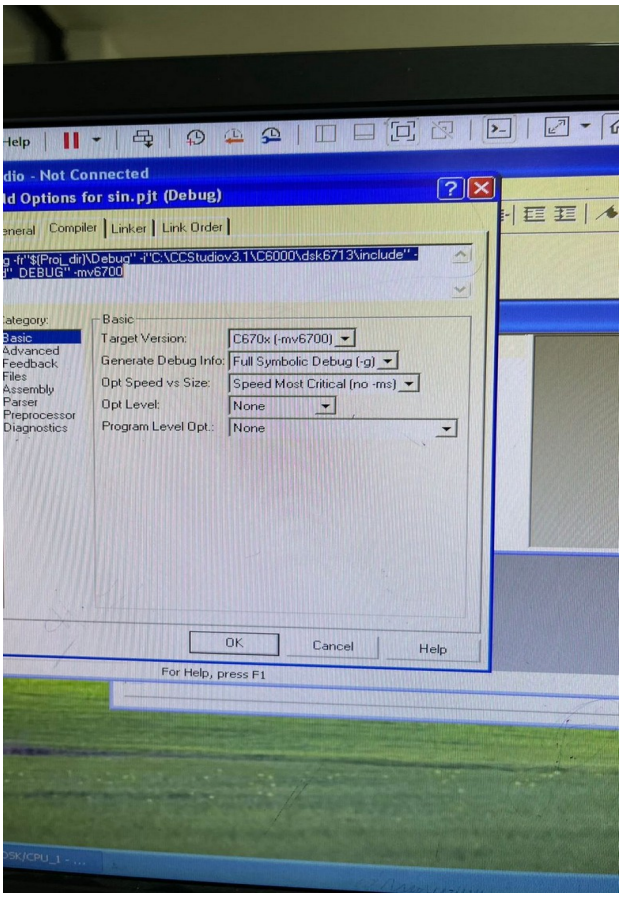
Verification

To verify if the kit is connected correctly with host and functioning properly “6713 DSK Diagnostic” is used.



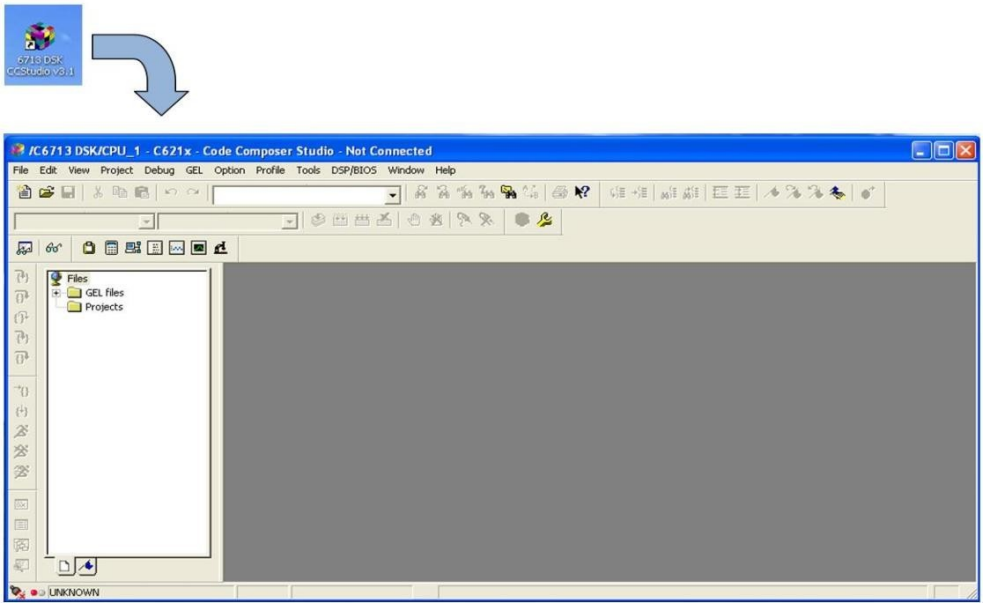
Board Connection Working





Creating a Project

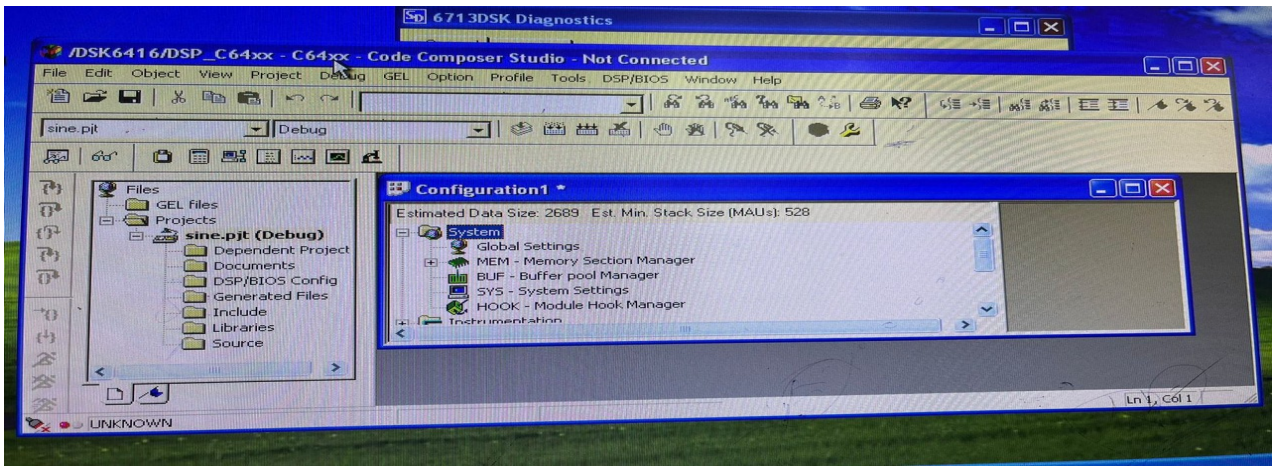
After verification of DSK6713 open Code Composer Studio v3.1. Following window will appear.



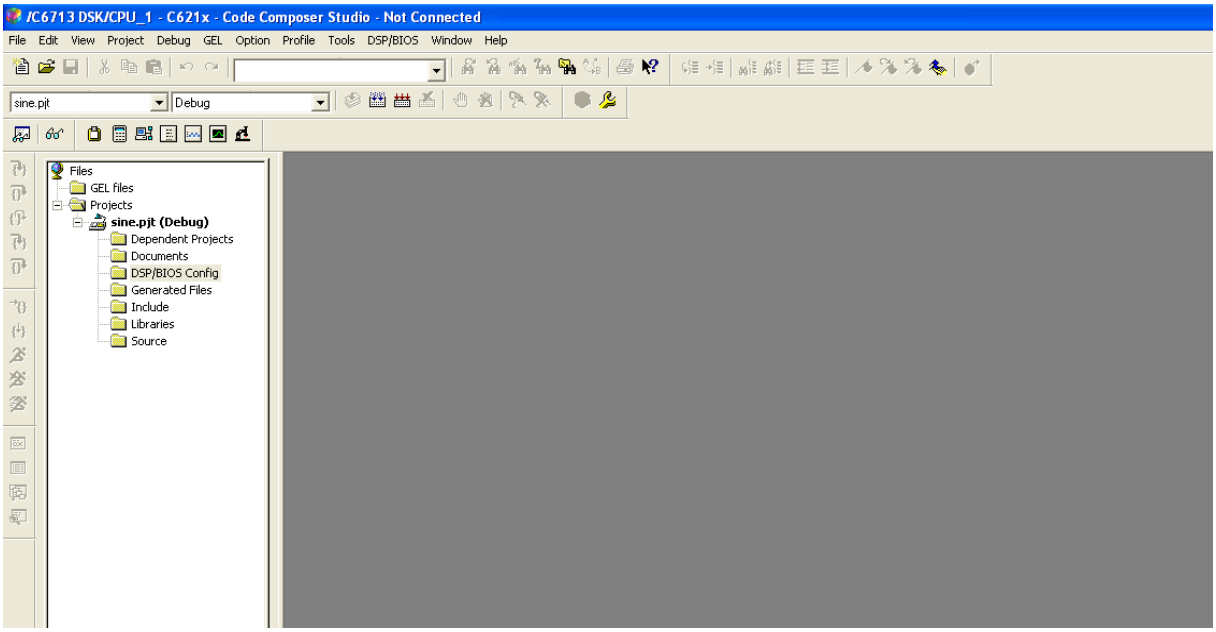
To create a new project, in the menu **Project New**

A window will open, type the name of Project and select the specific DSP kit you are working and press Finish. As shown below

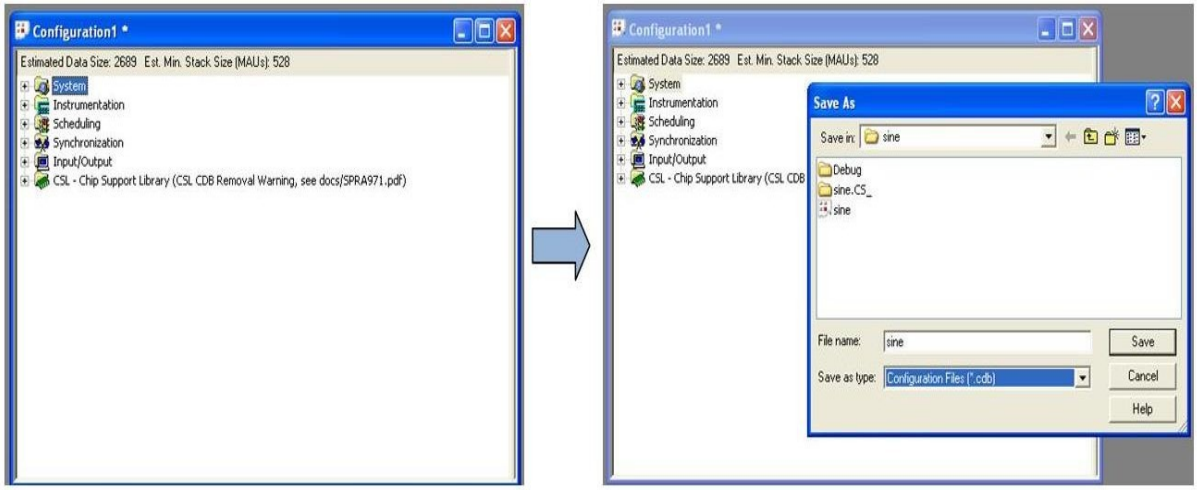
Project Setting SS



Window on the left side of CCS will appear like

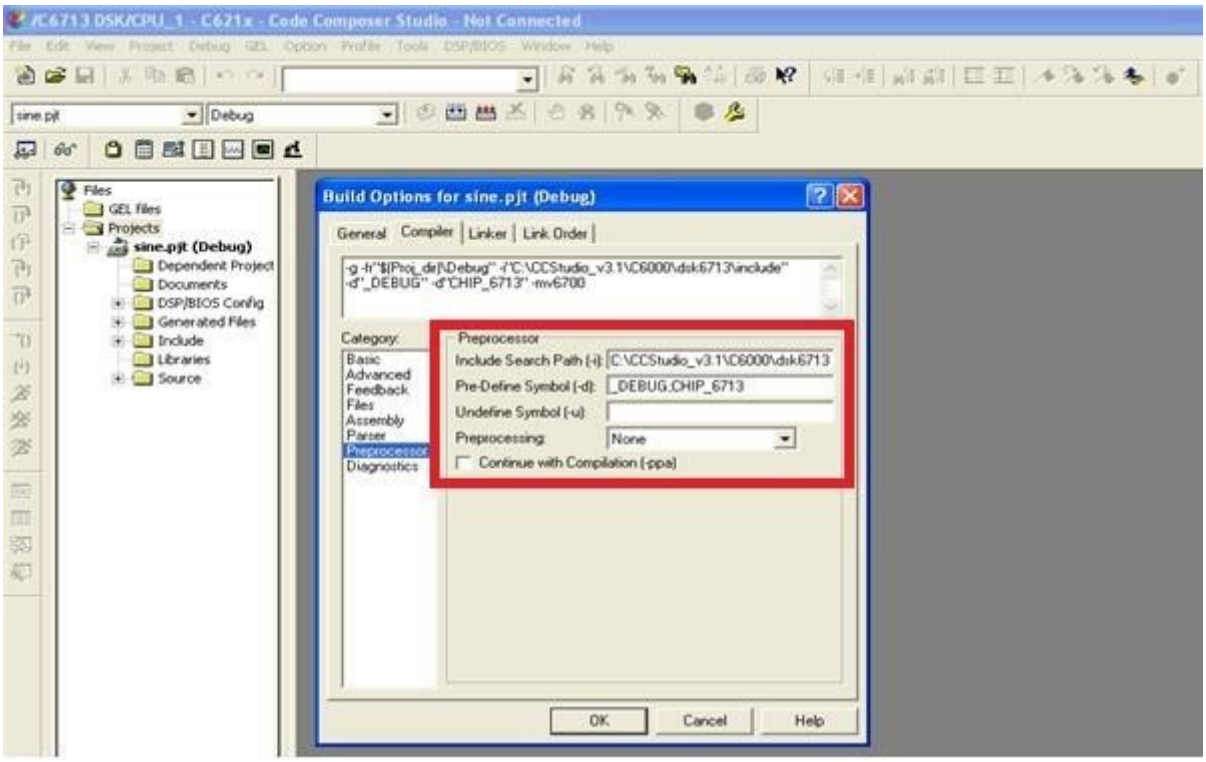


To set DSP BIOS Configuration, menu **File New DSP/BIOS Configuration.** A window will appear as configuration1. Save it with the same name as of your project in the project folder. As shown below.



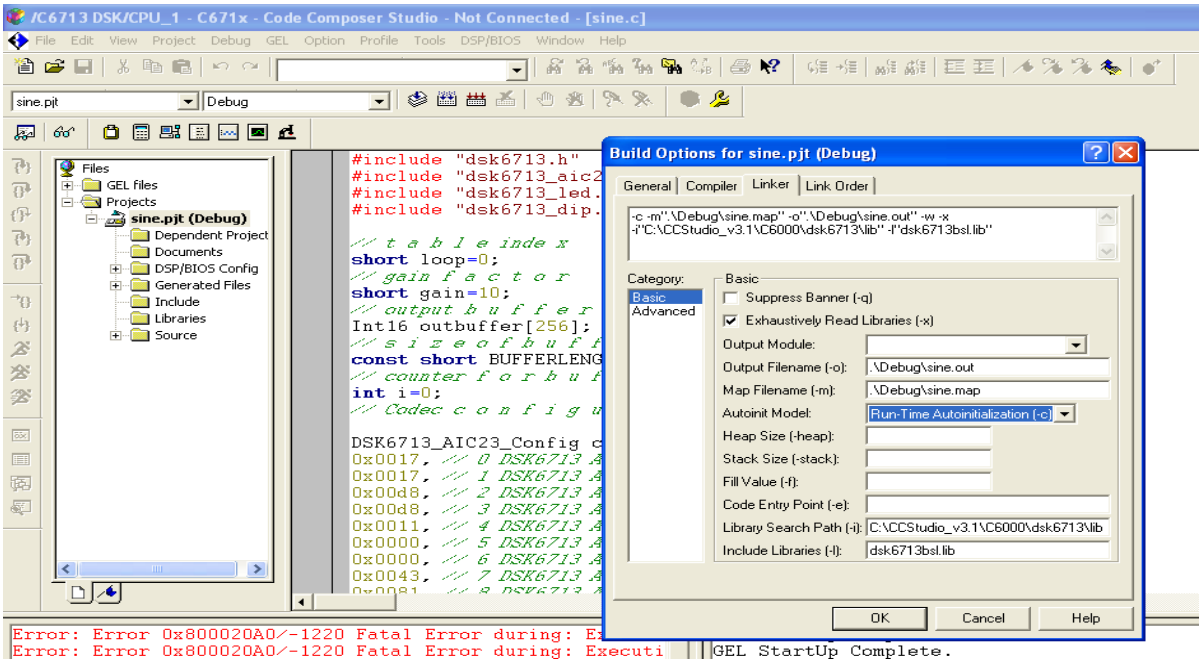
Load the BIOS configuration of DSK board in the project. Right click on **DSP/BIOS Configuration Add Files to Project** select the DSP/BIOS configuration file saved in above step.

In menu **Project Build Options.** A window will appear select **Compiler** category **Preprocessor** and enter the path of **dsk header** files (to load kit header files) and predefined symbol as DSP kit namei.e. CHIP_6713. It will look like as below



Note:*In general path of DSK header files is "C:\CCStudiov3.1\C6000\dsk6713\include".*

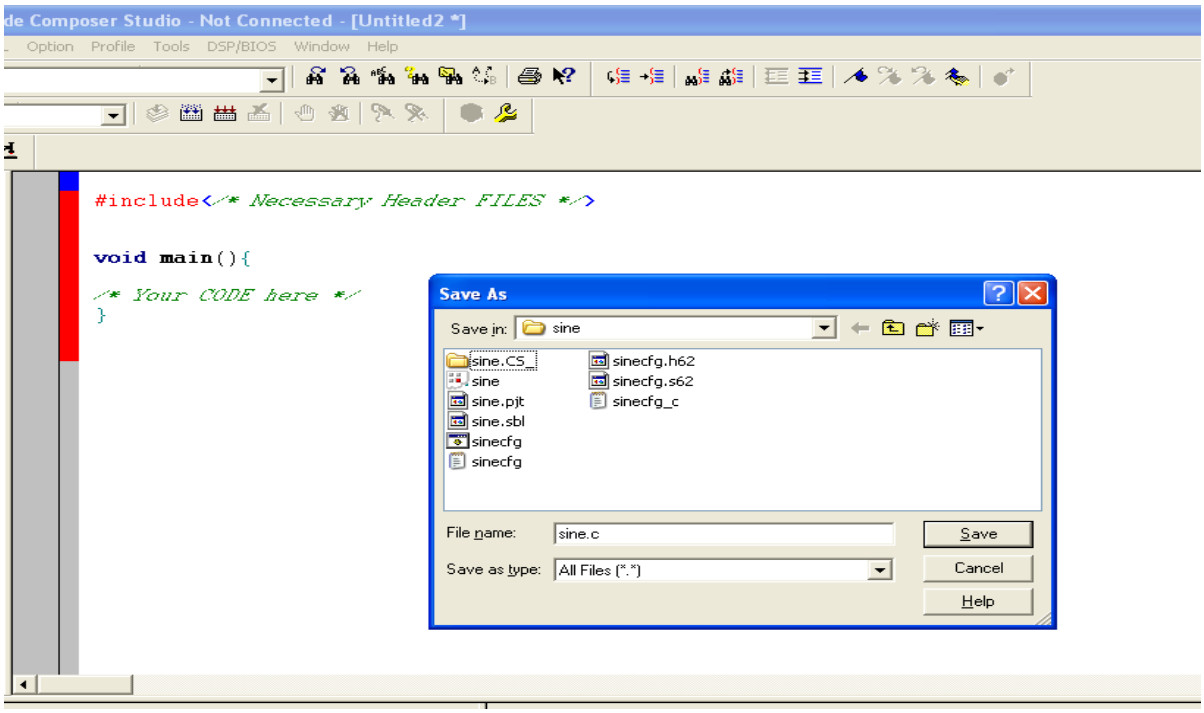
In menu**Linker**category **Basic** and set **Library Search Path** and **Include Libraries**.

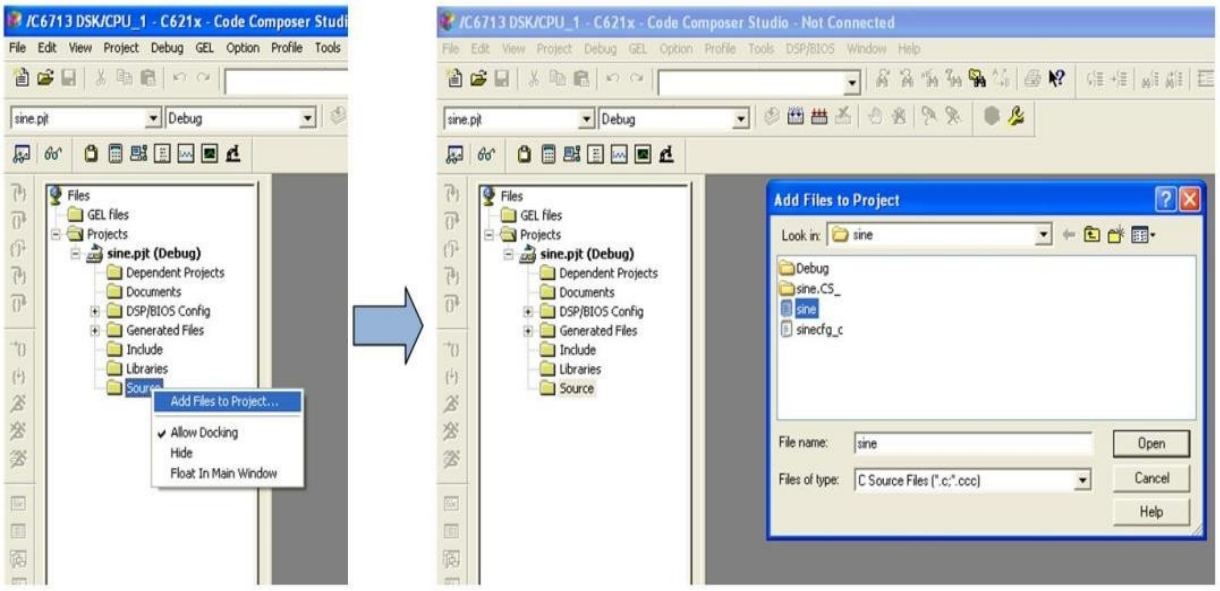


Note:*In general path of DSK Library files is "C:\CCStudiov3.1\C6000\dsk6713\lib". Libraries to include is "dsk6713bsl.lib"*

Writing Code in CCS

File New Source File. A window will open write your code there and save it in the Project Folder with the same name of Project with the extension "**.c**". As shown below

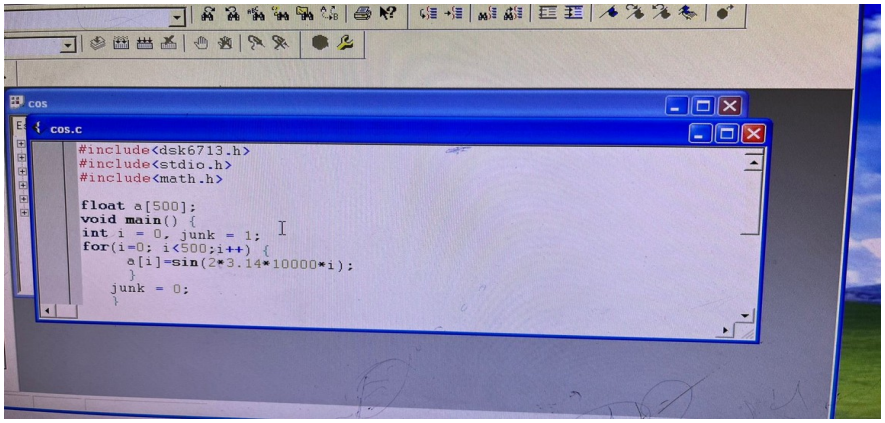




Load C Code to CCS. Right click on **Source** **Add Files to Project** select the C Code file saved before

Sinewave C Code

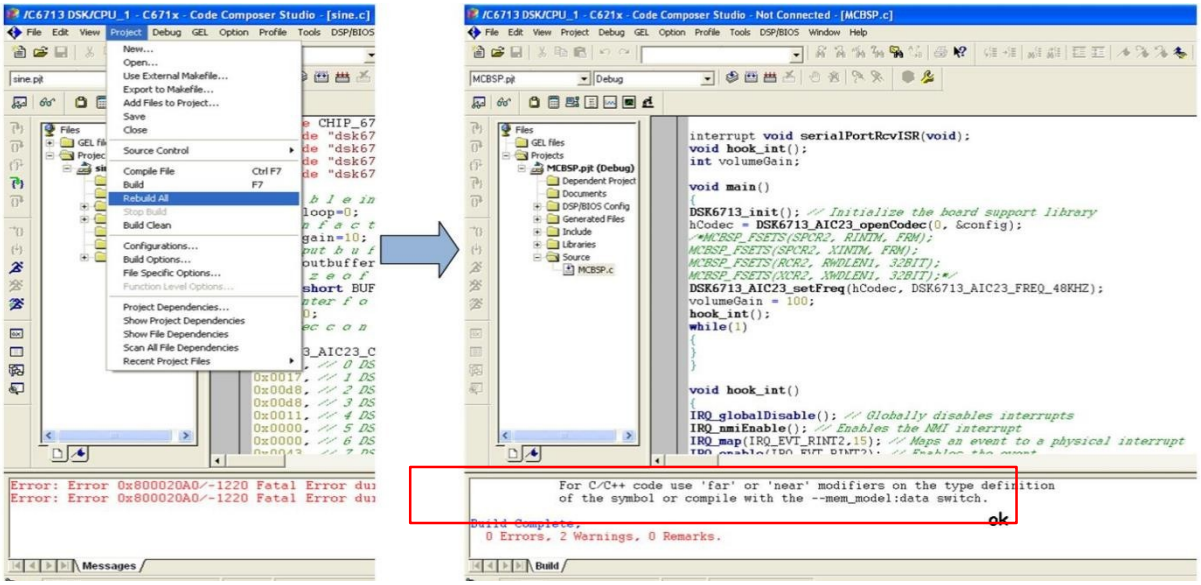
```
#include<dsk6713.h>
#include <stdio.h>
#include<math.h>
float a[500];
void main()
{ int i=0,
junk=1;
for(i=0;i<500;i++) {
        a[i]=sin(2*3.14*10000*i)
```



```
junk=0;
}
```

Build the Project

In menu **Project Build** or press **F7**.



If there is no Error in the code CCS IDE will create a hex file of extension **".out"**. Name of file will be same as of project.

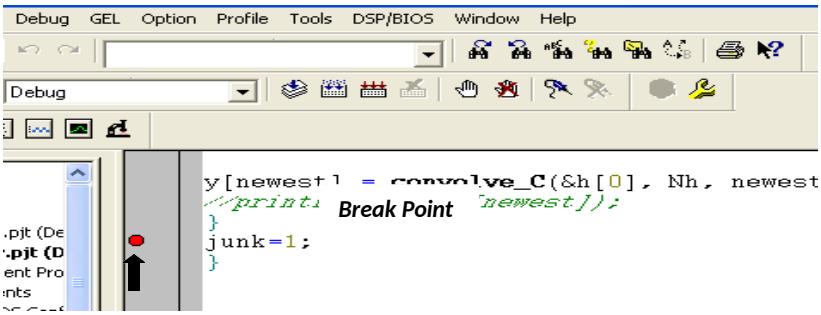
To run the project on **DSK6713,** connect CCS with DSK6713. In menu **Debug Connect** or press **Alt+C**.

- File Load Program ...: Select the ".out" file under debug folder.
- Debug Reset Cpu
- Debug Restart
- Debug Go Main
- Debug Run

To stop the processing menu **Debug Halt** or **Shift+F5.**

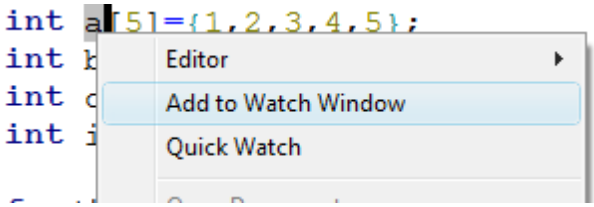
Plot Graphs

Code Composer Studio IDE provides a tool to plot figures in time and frequency domain to visualize the data. In this lab you will learn how to plot graphs.



Add to Watch Window

To see or change the values in variables, you can use the watch window. To add a variable to the watch window, right-click on the variable and select "Add to watch window"

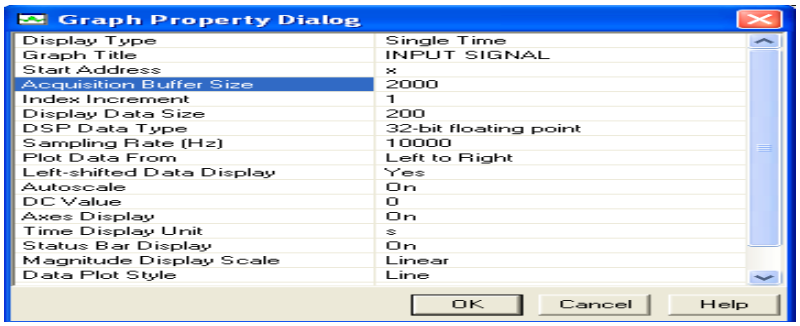


You can see the values and changes in the watch window at the bottom right of the CCS

Name	Value	Type	Radix
a	0x0000F034	int[5]	hex
[0]	1	int	dec
[1]	2	int	dec
[2]	3	int	dec
[3]	4	int	dec
[4]	5	int	dec

Plot Graph

Go to ***View > graph > time/frequency***



On the pop up window, fill:

- **Start Address:** Name of the array (to be plotted).
- **Acquisition Buffer Size:** Size of the array.
- **Display Data Size:** Size of the data to be displayed on the graph.
- **DSP Data Type:** Type of the array (integer or float).
- **Sampling Rate:** Sampling rate of the signal in the array.

To view spectrum of signal set Display Type to **"FFT Magnitude"**

Lab Task 2: Write a code to toggle led

Here are some functions to help you

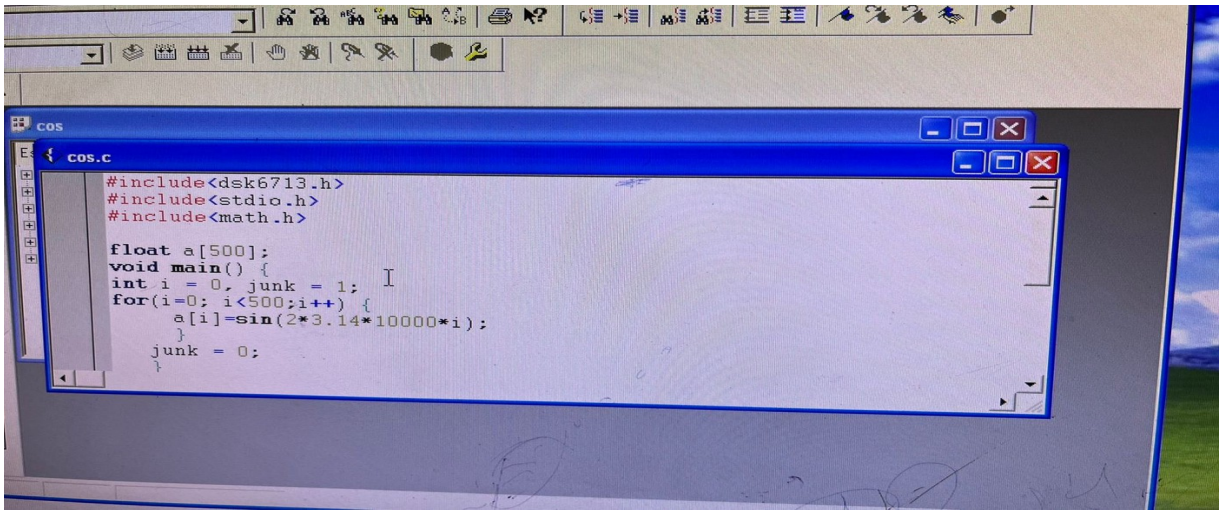
Function

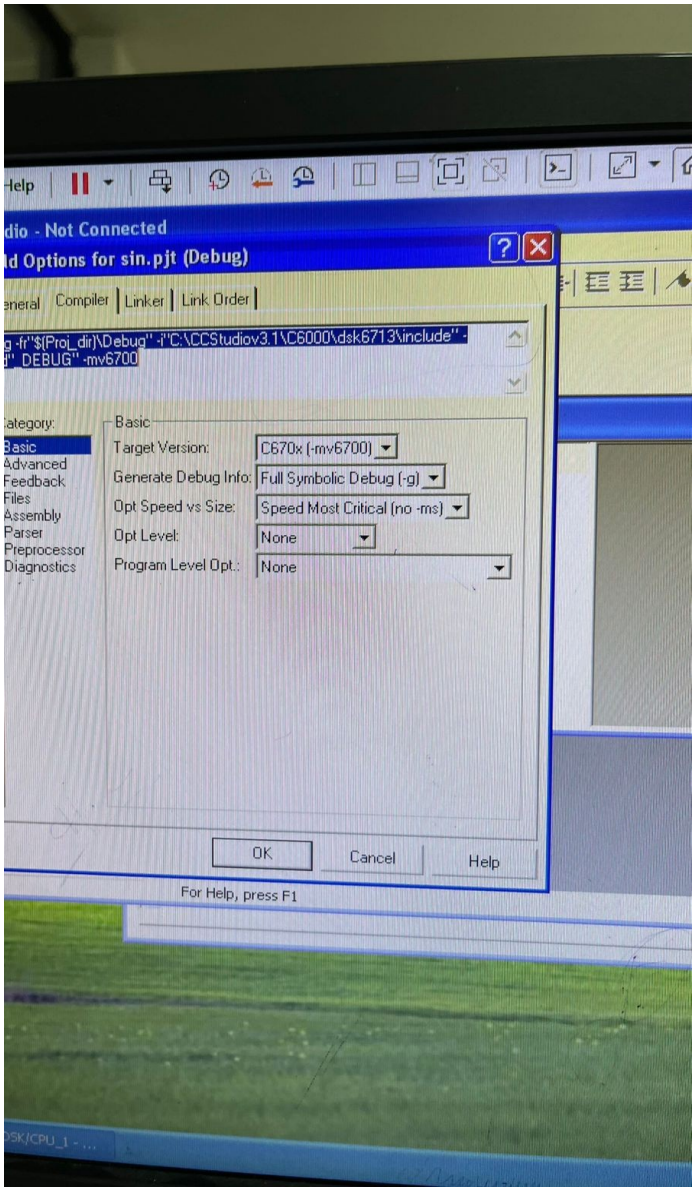
Details of C function require is given below

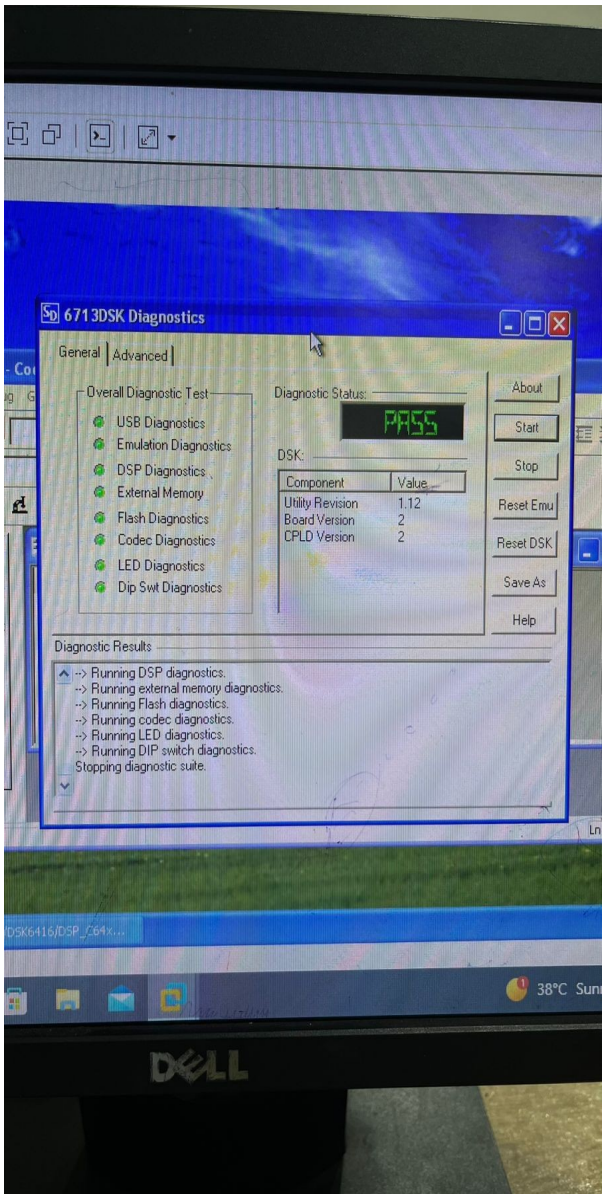
- DSK6713_init(); */* To Initialize DSP Starter Kit DSK6713 */*
  - Require header **dsk6713.h**
  - Input type Null
  - Return Type Null
  - To initialize DSK6713 kit

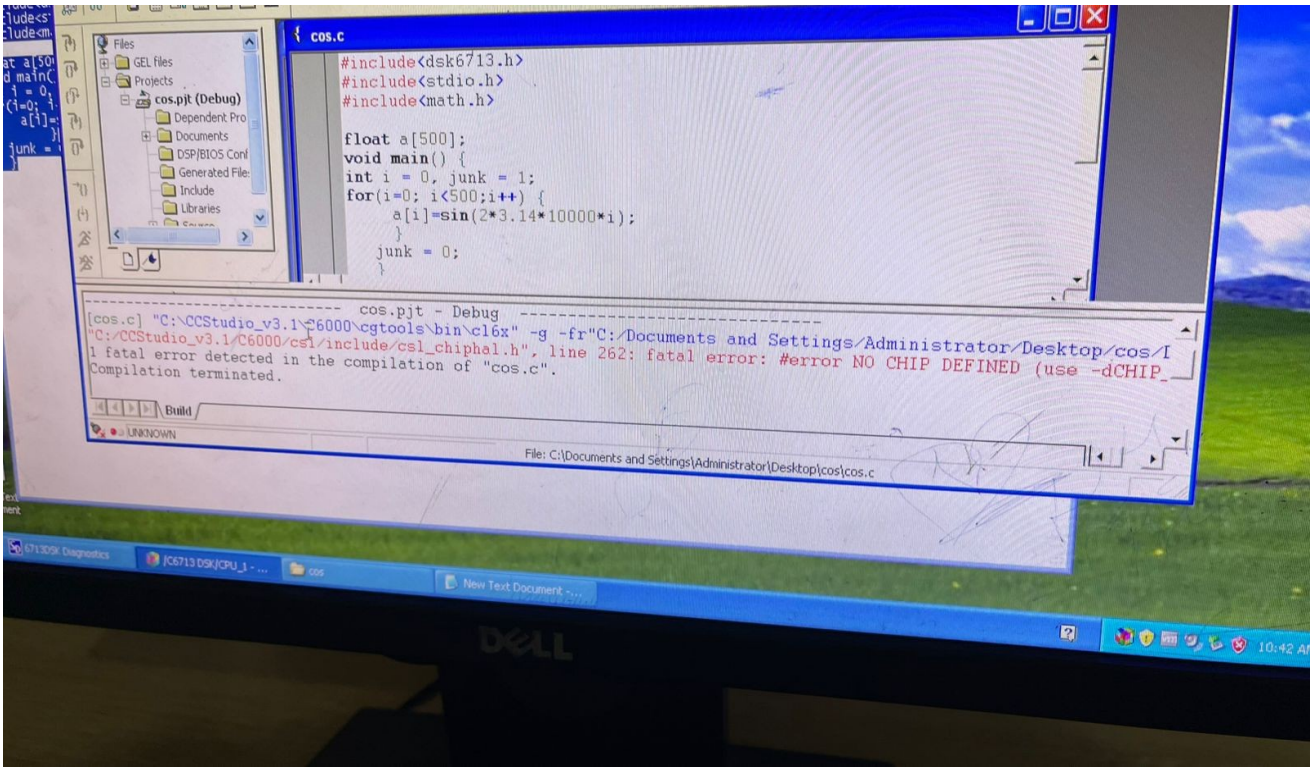
- DSK6713_LED_init(); */* To Initialize LEDs on DSK6713 */*
  - Require header **dsk6713_led.h**
  - Input type Null
  - Return Type Null
  - To initialize LEDs on DSK6713 kit
- DSK6713_DIP_init(); */* To Initialize DIP Switches on DSK6713 */*
  - Require header **dsk6713_dip.h**
  - Input type Null
  - Return Type Null
  - To initialize DIP switches on DSK6713 kit
- DSK6713_LED_on(int); */* To turn on specific LED on DSK6713 */*
  - Require header **dsk6713_led.h**
  - Input type integer (0-3)
  - Return Type Null
  - To turn on LED specified by input number DSK6713 kit
- DSK6713_LED_off(int); */* To turn on specific LED on DSK6713 */*
- Require header **dsk6713_led.h**
  - Input type integer (0-3)
  - Return Type Null
  - To turn off LED specified by input number DSK6713 kit
- int DSK6713_DIP_get(int); */* To check if switch on/off DSK6713 */*
  - Require header **dsk6713_dip.h**
  - Input type int
  - Return Type int */* Return 1 if switch is off and 0 if switch is on */*
  - To check the state of DIP switches on DSK6713 kit (return 0 when pressed, return 1 when not pressed)

Lab Working SnapShots for TASKS









Lab Task 3

Write a code to turn on LED

```
#include <dsk6713.h>
#include <dsk6713_led.h>

void main(void)
{
  // Initialize the DSK6713 board support library
  DSK6713_init();

  // Initialize the LED module
  DSK6713_LED_init();

  // Turn ON LED 0 (the first LED)
  DSK6713_LED_on(0);

  // Infinite loop to keep the LED ON
  while(1);
}
```

DSK6713_init(); → Initializes the DSK6713 board and Its peripherals.

DSK6713_LED_init(); → Sets up the LED control functions.

DSK6713_LED_on(0); → Turns ON LED 0 (you can change the number 0–3 for different LEDs).

while(1); → Keeps the program running so the LED remains ON continuously.

Lab Task 4

Write a code to turn on LED according to user instruction by DIP Switch

```
#include<dsk6713.h>
#include<dsk6713_led.h>

void main() {

/* To turn ON LED # 0 */
while(DSK6713_DIP_get(0))  {

        DSK6713_LED_on(0);

        }

/* To turn ON LED # 1*/
while(DSK6713_DIP_get(1))  {

        DSK6713_LED_on(1);

}
/* To turn ON LED # 2 */
while(DSK6713_DIP_get(2))  {

        DSK6713_LED_on(2);

}
/* To turn ON LED # 3 */
while(DSK6713_DIP_get(3))  {

        DSK6713_LED_on(3);

}
```

This program controls the LEDs on the TI DSK6713 DSP board using the DIP switches as user inputs. Each DIP switch corresponds to a specific LED. When the user turns ON a DIP switch, the program continuously checks its state using DSK6713_DIP_get(). If the switch is ON, the respective LED is turned ON using DSK6713_LED_on(). For example, when DIP switch 0 is ON, LED 0 lights up, and similarly for switches 1, 2, and 3. This setup allows the user to manually control which LEDs are turned ON or OFF in real time.

Rubric for Lab Assessment

The student performance for the assigned task during the lab session was:			
Excellent	The student completed assigned tasks without any help from the instructor and showed the results appropriately.	4	
Good	The student completed assigned tasks with minimal help from the instructor and showed the results appropriately.	3	
Average	The student could not complete all assigned tasks and showed partial results.	2	
Worst	The student did not complete assigned tasks.	1	

Instructor Signature: _____ Date: _____

LAB # 9

To display the basic operations for Audio Signal Processing using MATLAB Objectives

- To explain basics of audio signal processing using MATLAB
- To display aliasing phenomena in case of an audio signal using MATLAB
- To display echo phenomena in case of an audio signal using MATLAB

Introduction

Audio signal processing is an important application area of Digital Signal processing.We can use MATLAB to perform different operations related to audio signal processing. In this Lab we will go through some of the basic audio signal operations.We use the MATLAB command **audiorecorder** , to record audio data from an input device such as a microphone for processing in MATLAB. Similarly there is another command **recordblocking** that is used to record audio from an input device for the number of seconds specified (this time duration is passed as argument to recordblocking). If we want to get data of an audio file that is available in our computer we can use the command **audioread.** If we want to play an audio in MATLAB ,there are two commands :**sound** and **play**. The play command is more flexible and provides greater level of control as compared to **sound**. When we play an audio in MATLAB using **sound** command we cannot pause it or stop it until it is complete but in case of **play** command we can pause,resume and stop the audio before it is complete.

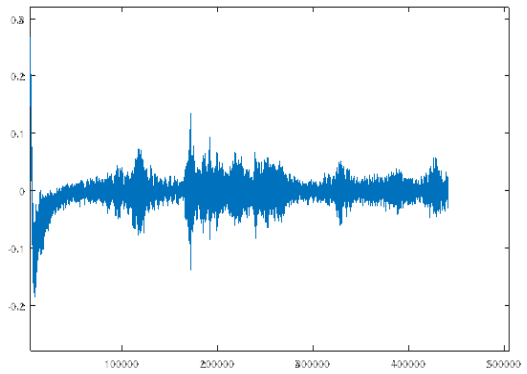
PreLab Tasks

Task 1: Recording an audio signal from an input device

Record an audio signal using the built in mike of our computer and then play that audio using MATLAB.

```
Fs = 44100;       nBits = 16;   nChannels = 1;
recObj = audiorecorder(Fs, nBits, nChannels);

disp('Start speaking...');
recordblocking(recObj, 10);
disp('Recording complete.');
audioData = getaudiodata(recObj);
disp('Playing recorded sound...');
sound(audioData, Fs);
figure;
plot(audioData);
```

```
Command Window
Start speaking...
Recording complete.
Playing recorded sound...
>>
```



Task 2: Reading an audio signal from your computer

Read an audio signal from any file available in your computer and then play that audio using MATLAB.

```
[y, Fs] = audioread('testingaudio.wav');
audio = audioplayer(y, Fs);
disp('Playing the audio file...');
play(audio);
figure
plot (y);
```

When program run, then testingaudio.wav audio will and also Graph will also plot.

InLab Tasks

Task 1: Demonstration of aliasing phenomenon.

We know that if sampling frequency is less than nyquist frequency ,aliasing will occur. Here in this task you will have to demonstrate aliasing for case of an audio signal. You may get the signal externally through an input device or you may read any audio file that is available on your computer.

```
info=audioread('testingaudio.wav');
[x,FS]=audioread('testingaudio.wav');
audio=audioplayer(x,FS)
play(audio);
audio=audioplayer(x,FS/2)
play(audio);
```

As sampling rate is half, (FS/2) , so Audio will play at slower speed or lower in pitch.

Task 2: Demonstration of echo phenomenon

Echo is a reflection of sound that arrives at the listener with a delay after the direct sound. Here in this task you will have to demonstrate echo phenomenon. You may get the signal externally through an input device or you may read any audio file that is available on your computer.

```
Fs = 44100;
audio = audioread('testingaudio.wav');
y = audioplayer(audio, Fs);
disp('Playing original audio...');
play(y);
pause(length(audio)/Fs + 1);
stop(y);

% adding echo
num = [1, zeros(1,10800), 0.9];
den = [1];
x = filter(num, den, audio);
y = audioplayer(x, Fs);
disp('Playing echoed audio...');
play(y);
pause(length(x)/Fs + 1);
stop(y);
pause(y);
resume(y);
stop(y);
```

This code demonstrates the echo phenomenon using an audio file. It first plays the original sound normally, then applies an echo effect by adding a delayed and scaled version of the same signal using a digital filter. The delay is created by inserting zeros equal to the number of samples for the desired echo t ime, and the scaling factor controls the echo's loudness. The filtered signal is then played to clearly hear the echo effect, (both signals can also be compared visually through waveform plots to observe the repetition caused by the echo).

```
>> LabTask2

Playing original audio...
Playing echoed audio...
>>
```

Rubric for Lab Assessment

The student performance for the assigned task during the lab session was:			
Excellent	The student completed assigned tasks without any help from the instructor and showed the results appropriately.	4	
Good	The student completed assigned tasks with minimal help from the instructor and showed the results appropriately.	3	
Average	The student could not complete all assigned tasks and showed partial results.	2	
Worst	The student did not complete assigned tasks.	1	

Instructor Signature: _____ Date: _____